

TECHNICAL REPORT

# Multi-Task and Objective Orchestration

## for Autonomous Mobile Robots

---

Ghada Ammar

2026

# Abstract

Autonomous mobile robots operating in dynamic, multi-user environments are increasingly required to manage complex, concurrent mission workloads under real-world operational constraints. Classical decision-making architectures, while reliable for predefined task sequences, lack the capacity to evaluate mission feasibility against resource limitations, or adapt to evolving environmental conditions limitations that become critical in service-intensive deployments.

This report presents the design and implementation of a multi-task orchestration system for an autonomous mobile robot, developed within the context of an engineering internship at Enova Robotics. The proposed system introduces a three-layer architecture that integrates environmental perception, mission-level evaluation, and reactive execution into a unified operational pipeline. It enables the robot to interpret operator instructions, assess task feasibility against live energy constraints, manage a dynamically ordered mission queue with priority-aware preemption, and navigate with the benefit of accumulated environmental knowledge.

This work suggests that combining environmental awareness with resource-conscious decision logic is a promising direction for improving robot autonomy in structured indoor environments. The results obtained support the feasibility of this approach and open perspectives for future extensions.

**Keywords :** · Multi-Task Orchestration · Mission Feasibility · Energy Constraints · Environmental Knowledge · Decision-Making · Dynamic Mission Queue

# Contents

|                                                                   |             |
|-------------------------------------------------------------------|-------------|
| <b>Abstract</b>                                                   | <b>i</b>    |
| <b>List of Acronyms</b>                                           | <b>viii</b> |
| <b>General Introduction</b>                                       | <b>1</b>    |
| <b>1 General Project Overview</b>                                 | <b>2</b>    |
| I Introduction . . . . .                                          | 2           |
| II Presentation of the Host Company . . . . .                     | 2           |
| 1 Organizational Structure . . . . .                              | 3           |
| 2 Services . . . . .                                              | 3           |
| III Presentation of the Project . . . . .                         | 4           |
| 1 Problem Statement . . . . .                                     | 4           |
| 2 Project Objectives . . . . .                                    | 4           |
| IV Methodology . . . . .                                          | 5           |
| 1 The Incremental Model . . . . .                                 | 5           |
| 1.1 Stages of the Incremental Model . . . . .                     | 5           |
| 1.2 Application of the Incremental Model to the Project . . . . . | 6           |
| V Conclusion . . . . .                                            | 6           |
| <b>2 Literature Review and Contextual Analysis</b>                | <b>7</b>    |
| I Introduction . . . . .                                          | 7           |
| II Theoretical Foundations and Technical Background . . . . .     | 7           |
| 1 Robot Operating System (ROS 2) . . . . .                        | 7           |
| 2 Simultaneous Localization and Mapping (SLAM) . . . . .          | 8           |
| 2.1 SLAM Toolbox . . . . .                                        | 9           |
| 3 Navigation Stack (Nav2) . . . . .                               | 10          |
| 3.1 Foundations of Autonomous Navigation . . . . .                | 10          |
| 3.2 Core Components of Navigation Stack . . . . .                 | 10          |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 4        | Robot Localization . . . . .                                             | 11        |
| 4.1      | Wheel Odometry . . . . .                                                 | 11        |
| 4.2      | Inertial Measurement Unit (IMU) . . . . .                                | 11        |
| 4.3      | Extended Kalman Filter (EKF) . . . . .                                   | 12        |
| 4.4      | Global Localization : Adaptive Monte Carlo Localization (AMCL) . . . .   | 12        |
| 4.5      | Synthesis : The Localization Hierarchy . . . . .                         | 13        |
| 5        | Differential Drive Robot . . . . .                                       | 14        |
| III      | Analysis of the Existing Solutions . . . . .                             | 14        |
| 1        | Classical Decision-Making Architectures in ROS 2-Based Systems . . . . . | 14        |
| 1.1      | Finite State Machines (FSM) . . . . .                                    | 14        |
| 1.2      | Behavior Trees (BT) . . . . .                                            | 15        |
| 1.3      | Comparative Analysis . . . . .                                           | 16        |
| IV       | Limitations and Critiques of the Existing Solutions . . . . .            | 16        |
| V        | Proposed Solution . . . . .                                              | 17        |
| 1        | Three Layer Architecture . . . . .                                       | 17        |
| 1.1      | Execution Layer . . . . .                                                | 17        |
| 1.2      | Decision Layer . . . . .                                                 | 17        |
| 1.3      | Memory Layer . . . . .                                                   | 18        |
| VI       | Conclusion . . . . .                                                     | 18        |
| <b>3</b> | <b>Requirements Specifications</b>                                       | <b>19</b> |
| I        | Introduction . . . . .                                                   | 19        |
| II       | Requirements Analysis . . . . .                                          | 19        |
| 1        | Functional Requirements . . . . .                                        | 19        |
| 2        | Non-Functional Requirements . . . . .                                    | 21        |
| III      | Requirement Specification . . . . .                                      | 22        |
| 1        | Actors Identification . . . . .                                          | 22        |
| 2        | Global Use Case Diagram . . . . .                                        | 22        |
| 3        | Use Cases Description . . . . .                                          | 24        |
| 3.1      | Use Case : Submit Mission . . . . .                                      | 24        |
| 3.2      | Use Case : Evaluate Feasibility . . . . .                                | 25        |
| 3.3      | Use Case : Dynamic Task Orchestration . . . . .                          | 26        |
| 3.4      | Use Case : Navigate to Objective . . . . .                               | 27        |
| IV       | Conclusion . . . . .                                                     | 28        |
| <b>4</b> | <b>Project Design</b>                                                    | <b>29</b> |
| I        | Introduction . . . . .                                                   | 29        |
| II       | Architectural Design . . . . .                                           | 29        |
| 1        | Global Architecture . . . . .                                            | 29        |
| III      | Detailed Architecture . . . . .                                          | 31        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 1        | Conceptual Design Foundations . . . . .                               | 31        |
| 1.1      | Task and Priority Model . . . . .                                     | 31        |
| 1.2      | Predictive Energy Forecasting . . . . .                               | 32        |
| 1.3      | Dynamic Preemption Model . . . . .                                    | 32        |
| 1.4      | Memory Integration . . . . .                                          | 32        |
| 2        | Structural Aspect . . . . .                                           | 33        |
| 3        | Behavioral Aspect . . . . .                                           | 33        |
| 3.1      | General Sequence Diagram . . . . .                                    | 33        |
| 3.2      | Memory injection sequence . . . . .                                   | 34        |
| 3.3      | Feasibility Evaluation sequence . . . . .                             | 35        |
| 3.4      | Delivery Preemption sequence diagram . . . . .                        | 36        |
| IV       | Conclusion . . . . .                                                  | 37        |
| <b>5</b> | <b>Project Realisation</b>                                            | <b>38</b> |
| I        | Introduction . . . . .                                                | 38        |
| II       | Development environment . . . . .                                     | 38        |
| 1        | Hardware Configuration . . . . .                                      | 38        |
| 2        | Technology Choices . . . . .                                          | 39        |
| 2.1      | Robot Operating System (ROS2) Jazzy . . . . .                         | 39        |
| 2.2      | Nav2 . . . . .                                                        | 39        |
| 2.3      | Behavior Tree . . . . .                                               | 39        |
| 2.4      | Python . . . . .                                                      | 39        |
| 2.5      | Ollama . . . . .                                                      | 39        |
| 2.6      | Langchain . . . . .                                                   | 40        |
| 2.7      | Vision Language Model Qwen3 . . . . .                                 | 40        |
| 2.8      | Milvus Db . . . . .                                                   | 40        |
| 2.9      | Streamlit . . . . .                                                   | 40        |
| 2.10     | Gazebo . . . . .                                                      | 40        |
| 2.11     | Rviz 2 . . . . .                                                      | 41        |
| III      | Implementation . . . . .                                              | 41        |
| 1        | Increment 1 : Autonomous Navigation and Simulation Baseline . . . . . | 41        |
| 1.1      | Objective . . . . .                                                   | 41        |
| 1.2      | Implemented Components . . . . .                                      | 41        |
| 2        | Increment 2 : Mission Parsing and Feasibility Evaluation . . . . .    | 43        |
| 2.1      | Objective . . . . .                                                   | 43        |
| 2.2      | Implemented Components . . . . .                                      | 43        |
| 3        | Increment 3 : Dynamic Task Orchestration . . . . .                    | 45        |
| 3.1      | Objective . . . . .                                                   | 45        |
| 3.2      | Implemented Components . . . . .                                      | 45        |

|                           |                                                               |           |
|---------------------------|---------------------------------------------------------------|-----------|
| 4                         | Increment 4 : Memory builder and Injection . . . . .          | 46        |
| 4.1                       | objective . . . . .                                           | 46        |
| 4.2                       | Implemented Components . . . . .                              | 46        |
| IV                        | Validation . . . . .                                          | 50        |
| 1                         | Environment Mapping . . . . .                                 | 50        |
| 2                         | Proximity-Based Task Reordering Validation . . . . .          | 51        |
| 3                         | high-priority Interrupt Validation . . . . .                  | 52        |
| 4                         | Obstacle Injection Validation . . . . .                       | 52        |
| 4.1                       | Blocked Corridor . . . . .                                    | 52        |
| 4.2                       | Crowded Corridor . . . . .                                    | 53        |
| V                         | Conclusion . . . . .                                          | 54        |
| <b>General Conclusion</b> |                                                               | <b>55</b> |
| <b>Bibliography</b>       |                                                               | <b>56</b> |
| <b>Appendices</b>         |                                                               | <b>A</b>  |
|                           | Appendix 1 : ROSbot 2 Pro Technical Specifications . . . . .  | A         |
|                           | Appendix 2 : Mission Parser EXTRACTION PROMPT . . . . .       | B         |
|                           | Appendix 3 : VLM Pass 1 Scene Classification Prompt . . . . . | C         |
|                           | Appendix 4 : VLM Pass 2 Obstacle Grounding Prompt . . . . .   | D         |

# List of Figures

|     |                                                                                       |    |
|-----|---------------------------------------------------------------------------------------|----|
| 1.1 | Enova Robotics Logo . . . . .                                                         | 2  |
| 1.2 | Enova Robotics organizational chart . . . . .                                         | 3  |
| 1.3 | Incremental model methodology process[2]. . . . .                                     | 6  |
| 2.1 | Communication modes in ROS 2 [3] . . . . .                                            | 8  |
| 2.2 | Differential drive robot concept[11]. . . . .                                         | 14 |
| 2.3 | Example of a Finite State Machine for robotic task execution[13]. . . . .             | 15 |
| 2.4 | Example structure of a Behavior Tree [15]. . . . .                                    | 15 |
| 3.1 | Global Use Case Diagram. . . . .                                                      | 22 |
| 4.1 | Global System Architecture Diagram. . . . .                                           | 30 |
| 4.2 | Class diagram . . . . .                                                               | 33 |
| 4.3 | Nominal Flow Sequence Diagram . . . . .                                               | 34 |
| 4.4 | Memory Injection Sequence Diagram . . . . .                                           | 35 |
| 4.5 | Feasibility Evaluation sequence Diagram . . . . .                                     | 36 |
| 4.6 | Delivery Preemption sequence Diagram . . . . .                                        | 37 |
| 5.1 | 3D visualisation of the hospital simulation environment . . . . .                     | 50 |
| 5.2 | Costmap generated by the navigation stack for autonomous navigation and path planning | 51 |
| 5.3 | UI response showing reordered chain mission based on proximity . . . . .              | 51 |
| 5.4 | UI response showing delivery preemption with resume order . . . . .                   | 52 |
| 5.5 | Blocked corridor scenario and corresponding costmap modification. . . . .             | 53 |
| 5.6 | Crowded corridor scenario and corresponding costmap modification. . . . .             | 54 |
| A.1 | Rosbot 2 pro Datasheet. . . . .                                                       | A  |
| B.1 | LLM Extraction Prompt . . . . .                                                       | B  |
| C.1 | VLM pass 1 Prompt . . . . .                                                           | C  |
| D.1 | VLM pass 2 prompt . . . . .                                                           | D  |

# List of Tables

|     |                                                                                    |    |
|-----|------------------------------------------------------------------------------------|----|
| 2.1 | Nav2 Stack Components [7]. . . . .                                                 | 11 |
| 2.2 | Comparison between Finite State Machines (FSMs) and Behavior Trees (BTs) . . . . . | 16 |
| 3.1 | Functional Requirements . . . . .                                                  | 20 |
| 3.2 | Non-Functional Requirements of the Proposed System . . . . .                       | 21 |
| 3.3 | Global Use Case Descriptions . . . . .                                             | 23 |
| 3.4 | Textual Description of UC-01 : Submit Mission . . . . .                            | 24 |
| 3.5 | Textual Description of UC-02 : Negotiate Feasibility . . . . .                     | 25 |
| 3.6 | Textual Description of UC-03 : Dynamic Task Orchestration . . . . .                | 26 |
| 3.7 | Textual Description of UC-04 : Navigate to Objective . . . . .                     | 27 |
| 4.1 | Preemption Mechanisms . . . . .                                                    | 32 |
| 5.1 | Hardware Specifications of the Development Environment . . . . .                   | 38 |
| 5.2 | Costmap Value Interpretation . . . . .                                             | 42 |
| 5.3 | OAK-D Camera Intrinsic Parameters . . . . .                                        | 47 |
| 5.4 | Milvus Memory Record Schema . . . . .                                              | 49 |
| 5.5 | Memory Injection Types and Costmap Effects . . . . .                               | 49 |

# List of Acronyms

|              |                                         |
|--------------|-----------------------------------------|
| <b>ARQ</b>   | : Attentive Reasoning Queries           |
| <b>BT</b>    | : Behavior Tree                         |
| <b>EKF</b>   | : Extended Kalman Filter                |
| <b>FSM</b>   | : Finite State Machine                  |
| <b>LiDAR</b> | : Light Detection and Ranging           |
| <b>LLM</b>   | : Large Language Model                  |
| <b>Nav2</b>  | : Navigation 2                          |
| <b>ROS</b>   | : Robot Operating System                |
| <b>SLAM</b>  | : Simultaneous Localization and Mapping |
| <b>UI</b>    | : User Interface                        |
| <b>VLM</b>   | : Vision-Language Model                 |

# General Introduction

Autonomous mobile robots are today deployed across an expanding spectrum of high-stakes environments : logistics facilities, public infrastructure, industrial sites, and service-critical spaces where operational continuity is not a convenience but a requirement. In these contexts, reliable navigation is not the challenge it is the baseline. The true measure of a robot's operational value lies upstream of locomotion, in its capacity to reason : to receive a stream of mission requests, assess their feasibility against available resources, establish a coherent execution order, and revise that order as the situation evolves.

This is the problem of autonomous task orchestration. Each mission does not arrive in isolation. It arrives alongside others, in a changing environment, from an operator, at a moment when the robot's own resources may already be partially committed. Managing this complexity demands more than a well-tuned navigation stack. It demands a system capable of understanding its operational context, not merely reacting to it.

Classical decision-making architectures offer deterministic execution of predefined sequences a valuable property in controlled environments. However, they cannot evaluate energy feasibility, infer intent from ambiguous instructions, weigh competing objectives, or retain environmental knowledge. In dynamic, real-world operations, these limitations are decisive.

This project, conducted at Enova Robotics, is a contribution toward closing that gap. It addresses the design and implementation of a task orchestration system for an autonomous mobile robot, capable of managing multi-mission workflows under real-world constraints. The work pursues three objectives : designing an environmental awareness perception layer, implementing a resource-conscious decision making engine that evaluates mission feasibility, and developing a dynamic priority management mechanism.

This report is structured into five chapters.

- **Chapter 1** presents the host company, project context, and development methodology.
- **Chapter 2** surveys the theoretical foundations and analyses existing approaches, identifying the limitations that motivate a new approach.
- **Chapter 3** formalises the system requirements.
- **Chapter 4** presents the architectural design of the proposed solution.
- **Chapter 5** details the implementation and validation results.

**Keywords :** *Autonomous Mobile Robot · Task Orchestration · Resource Conscious · Environmental Awareness · Dynamic Priority Management*

# General Project Overview

## I Introduction

This chapter presents a clear and comprehensive overview of the project, highlighting its context and scope. We begin by introducing the host company. Then we define the main objectives of the project and outline the methodology used to achieve them.

## II Presentation of the Host Company

This internship was carried out within Enova Robotics, a Tunisian technology company founded in 2014 by Dr. Anis Sahbani and located at Novation City, Sousse, specializing in advanced security technology that leverages software and hardware to provide its clients with cutting-edge anomaly detection capabilities. Enova Robotics brings together unique expertise in mobile robotics, artificial intelligence, and security engineering, allowing it to design highly specialized products that represent the future of the sector. As the security and surveillance industry undergoes a significant transformation driven by the rise of autonomous systems, Enova Robotics positions itself at the forefront of this shift, delivering solutions that are more efficient, more reliable, and better adapted to the complex demands of modern security environments.

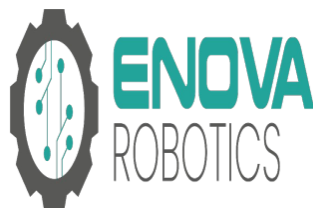


FIGURE 1.1 – Enova Robotics Logo

Having outlined the companys profile, the following sections examine its organizational structure, key divisions, and service offerings .

## 1 Organizational Structure

Enova Robotics operates under the direct leadership of its CEO and Founder, who oversees five core departments :

- **VP Sales** : Responsible for commercial development and client relationship management.
- **VP Industry** : Oversees production operations through two dedicated functions a Production Supervisor and a Procurement unit.
- **Logistics** : Ensures the timely and reliable distribution of goods and systems to clients.
- **VP Research & Development** : Organized into four specialized hubs Full Stack, IA, Navigation, and Mechatronics. The present internship was carried out within this department.
- **Design** : Translates client requirements into viable product architectures under real-world operational constraints.
- **Financial Management** : Oversees budgetary planning and financial operations across the organization.
- **Quality Management** : Ensures compliance with quality standards at every stage of production and delivery.
- **Human Resources** : Manages recruitment, personnel administration, and organizational development.
- **IT** : Maintains the company’s technological infrastructure and internal systems.

The complete hierarchical structure of the organization is depicted in Figure 1.2.

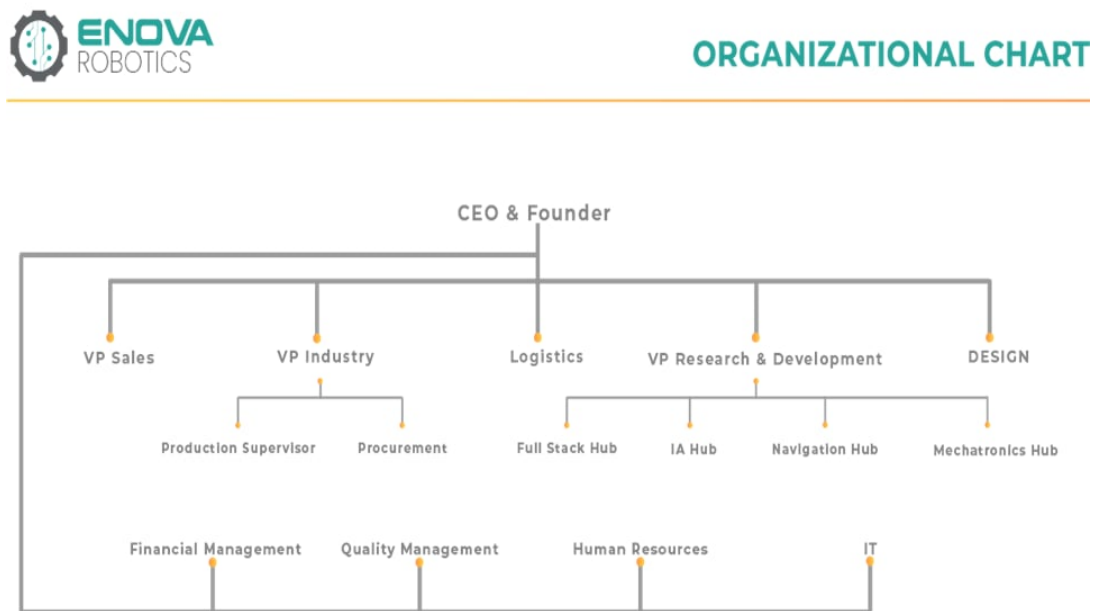


FIGURE 1.2 – Enova Robotics organizational chart

## 2 Services

Enova Robotics operates as an end-to-end robotics solutions provider, accompanying clients across the full development spectrum from early ideation through to the delivery of fully operational systems.

Its activities are organized around three core pillars :

- **Design** : Client requirements are translated into concrete product architectures under real-world operational constraints, with iterative refinement until functional maturity is reached.
- **Research and Development** : Enova Robotics conducts applied research for clients requiring purpose-built systems while advancing its own proprietary product line, maintaining a competitive edge in a rapidly evolving field.
- **Manufacturing** : Hardware assembly, system integration, and pre-deployment testing are handled entirely in-house, ensuring full traceability and quality assurance at every production stage.

### III Presentation of the Project

#### 1 Problem Statement

Autonomous mobile robots deployed in real world environments frequently face the challenge of performing multiple tasks and achieving several objectives within a single mission, such as exploration, multi-point navigation, or context-dependent actions. Traditional control systems use fixed, preprogrammed pipelines that cannot handle dynamic environments, changing mission priorities, or human instructions. As a result, robots have difficulty adapting when conditions change, which reduces their autonomy and mission reliability.

Beyond the need for adaptability, modern robotic applications increasingly require interaction and negotiation with human operators. Robots must not only execute tasks, but evaluate conflicting objectives, manage operational constraints (e.g., battery, risks), and determine whether to accept or reject new assignments based on feasibility. This transforms mission planning into a decision-making problem, where the robot must balance priorities, interpret human intent, and dynamically reconfigure its behavior.

These challenges create a clear need for a robust, high-level task orchestration system that integrates decision-making, evaluation capabilities, and spatio-temporal memory, enabling robots to assess multiple objectives and adjust their behavior in real time.

#### 2 Project Objectives

The goal of this project is to design and implement a multi-task orchestration system for an autonomous mobile robots. This system explicitly addresses the challenges of decision making, and human robot interaction. The project aims to :

- Develop an architecture capable of sequencing, prioritizing, and deciding on multiple tasks and objectives in dynamic environments.
- Evaluate operational constraints, such as battery and priority, to guarantee reliable robot behavior.
- Enable the robot to interpret human instructions and assess assignments.
- Provide real-time adaptation to environmental changes and evolving mission requirements, supported by spatio-temporal memory.

## IV Methodology

The development of this project which integrates multiple evolving components , the requirements could not be fully defined at the outset. Each component's interface and behavior could only be validated after observing the output of the previous one. This reality called for a development approach that supports progressive construction, early delivery of functional subsystems, and continuous adjustment as the system evolves.

The Incremental Model was therefore selected as the development methodology for this project. In the following section, we provide an overview of the Incremental Model.

### 1 The Incremental Model

The Incremental Model is an SDLC approach in which functionality is delivered through a series of discrete, plan-driven increments, each extending the previous release until the complete system is produced. It is well suited to a single developer working within a time-bounded project, as each increment carries a clearly defined scope and a concrete deliverable.[1],[2]

#### 1.1 Stages of the Incremental Model

Each increment proceeds through five stages [1],[2] :

- **Planning** : the system is decomposed into increments, each assigned a defined scope and expected outcome.
- **Analysis and Design** : requirements for the current increment are elicited and a design solution is formulated within the constraints of prior increments.
- **Implementation** : the increment is developed and unit-tested, extending the existing system without disrupting prior functionality.
- **Integration and Testing** : the increment is consolidated with previous work and system-level verification confirms correct behaviour across all components.
- **Deployment** : the integrated system is released as a functional intermediate deliverable, serving as the baseline for the next increment.

The following figure 1.3 illustrates the incremental model methodology process.

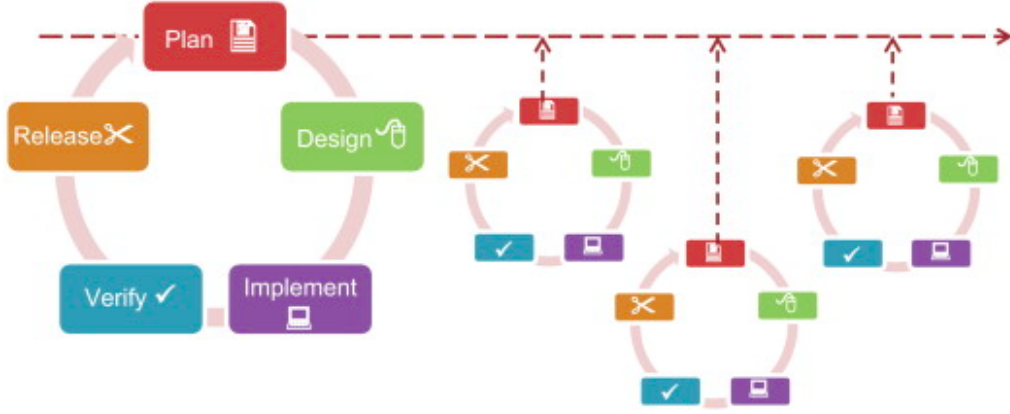


FIGURE 1.3 – Incremental model methodology process[2].

## 1.2 Application of the Incremental Model to the Project

The project was developed following an incremental approach, where each increment represents a major step in the system development process. The following list describes the main increments and their respective objectives.

- **Increment 1** : Setup of the robotic platform and simulation environment.
- **Increment 2** : Evaluation of mission requests based on the robot's available resources.
- **Increment 3** : Management and adaptation of accepted missions within an evolving schedule.
- **Increment 4** : Integration of environmental knowledge to support mission execution.

## V Conclusion

In this chapter, we provided an overview of Enova Robotics, presenting the project context, objectives, and adopted methodology, emphasizing the objective of enhancing the robots capabilities. Moving forward, we will develop and implement the robotic system, integrating relevant concepts to address the challenges posed by this project.

# Literature Review and Contextual Analysis

## I Introduction

This chapter is essential for establishing a solid understanding of the projects theoretical background and for examining existing solutions. It begins with a Background overview in robotics, followed by an analysis and comparison of existing approaches then highlights their limitations , emphasizing the need for improvement.

## II Theoretical Foundations and Technical Background

This section presents the key concepts and technologies forming the basis of this work. It covers the core robotic middleware and navigation components : ROS 2, SLAM, the Nav2 navigation stack, and robot localization, as well as the kinematic model used, before addressing the higher-level building blocks specific to this work.

### 1 Robot Operating System (ROS 2)

ROS 2 is an open-source middleware framework comprising a rich collection of software libraries and development tools designed to support the construction of robotic applications across a wide variety of hardware platforms. It provides a structured execution environment specifically tailored to the demands of robotic systems, offering ready-to-use implementations of common algorithms, simulation utilities, and hardware-interface abstractions. The following paragraphs introduce the core architectural concepts of ROS 2 and describe its primary communication mechanisms.[3]

- **Nodes :** In ROS 2, a node is the fundamental computational unit responsible for carrying out a well-defined, self-contained function, such as reading sensor data, running a control algorithm, or driving an actuator. Each node operates as an independent process, and a complete robotic application is typically composed of multiple cooperating nodes. Nodes are organized into package, where each package bundles one or more nodes that collectively address a specific functional concern.[3]

- **Topics** : Inter-node communication in ROS 2 relies primarily on topics, which implement an asynchronous publish/subscribe messaging model. A topic constitutes a named, typed channel over which one or more publisher nodes continuously broadcast data, such as sensor measurements or velocity references, while any number of subscriber nodes independently receive those messages. A given node may act as either a publisher or a subscriber on a particular topic, but not both simultaneously. There is otherwise no restriction on the number of nodes participating on either side.[3]
- **Services** : For interactions that require a direct, bidirectional exchange, ROS 2 provides services, which follow a synchronous clientserver paradigm. A server node registers a named service and waits for incoming requests; any client node may then invoke that service and block until a response is returned. Unlike topics, which support unidirectional data streaming, services permit the client to attach structured data to the request itself, enabling a tightly coupled, queryresponse communication pattern.[3]

To illustrate the different communication mechanisms supported by ROS 2, Figure 2.1 presents the principal interaction modes available between nodes.

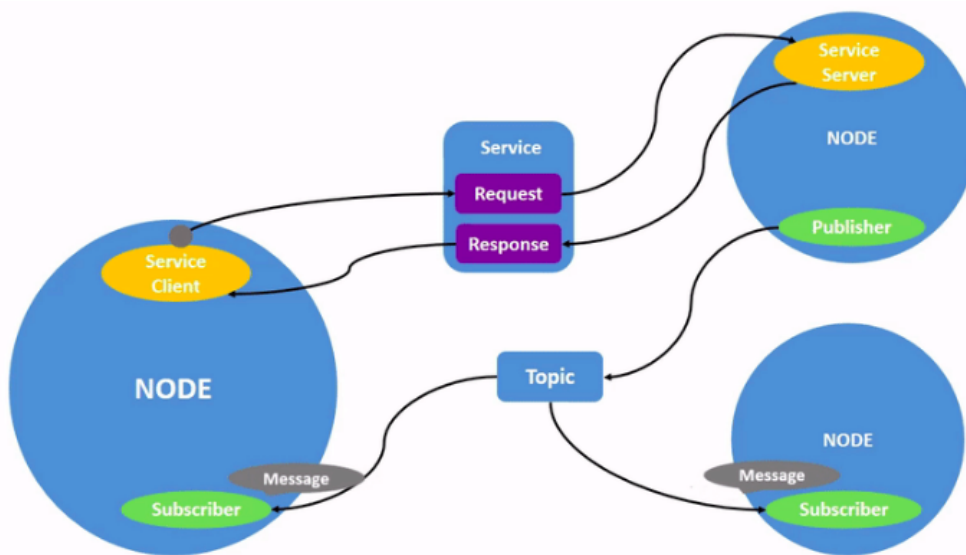


FIGURE 2.1 – Communication modes in ROS 2 [3]

With the communication infrastructure in place, the robot must next acquire a geometric understanding of its environment. This is achieved through Simultaneous Localization and Mapping, presented in the following section.

## 2 Simultaneous Localization and Mapping (SLAM)

Simultaneous Localization and Mapping (SLAM) is the process through which a mobile robot constructs a map of an unknown environment while concurrently tracking its own position within that map, making it a foundational capability for autonomous navigation in previously unseen spaces.[4]

In practice, SLAM relies on two complementary data sources : a LiDAR scanner, which captures

the surrounding geometry, and an odometry system, which estimates the robot’s displacement from wheel encoder measurements. Both sources are imperfect. SLAM algorithms address this by continuously reconciling new observations against the evolving map, preserving its geometric consistency[4]. Within ROS 2, this is achieved through Slam Toolbox, presented in the following section.

## 2.1 SLAM Toolbox

Within the ROS 2 ecosystem, SLAM Toolbox is a widely adopted, high-performance mapping solution distinguished by its support for lifelong and multi-session mapping scenarios and its extensible plugin-based architecture. The framework relies on advanced scan-matching and loop-closure algorithms to maintain map accuracy : whenever the robot revisits a previously mapped region, the system identifies the correspondence and triggers a global map correction. The map-building pipeline encompasses the following sequential stages [5] :

1. **Data Acquisition.** As the robot moves through the environment, an onboard LiDAR scanner continuously captures distance measurements of the surroundings, while wheel encoders supply incremental displacement estimates.[5]
2. **Scan Matching.** Each incoming LiDAR scan is aligned against previously recorded scans to estimate the robot’s relative pose increment, progressively refining the odometric trajectory.[5]
3. **Pose-Graph Optimization.** The sequence of robot poses estimated over time, together with the inter-pose transformations derived from scan matching, are encoded as a pose graph in which nodes represent poses and edges encode spatial constraints. As new observations are incorporated, the graph is subject to non-linear optimization, redistributing accumulated errors uniformly across the entire trajectory to produce a globally consistent map.[5]
4. **Loop Closure Detection and Correction.** A critical mechanism within SLAM Toolbox is the detection of loop closures, which occurs when the robot re-enters a location it has previously visited. Upon recognizing such an event, the system quantifies the accumulated trajectory drift and propagates a corrective adjustment back through the pose graph, simultaneously refining all previously estimated poses and their associated map contributions. This correction substantially improves map accuracy, particularly in large-scale environments where drift would otherwise compound over extended traversals.[5]
5. **Occupancy Grid Generation.** From the set of optimized robot poses and their corresponding aligned laser scans, SLAM Toolbox generates a 2D occupancy grid map. Each cell in this discrete spatial representation encodes a probabilistic estimate of occupancy : cells may be classified as occupied by an obstacle, confirmed as free space, or designated as unexplored.[5]

A consistent map alone is insufficient for autonomous operation : the robot must also be capable of planning and executing collision-free trajectories within that map. This is precisely the role of the Nav2 navigation stack, described next.

### 3 Navigation Stack (Nav2)

The Navigation Stack (Nav2) framework is a modular, server-based navigation stack for ROS 2 that provides mobile robots with the full pipeline required for autonomous point-to-point navigation, including map-based path planning, real-time obstacle avoidance, and recovery from failure states.[6]

#### 3.1 Foundations of Autonomous Navigation

Autonomous navigation is the capacity of a robot to independently compute and execute a sequence of motion commands that bring it from its current pose to a desired target location, while guaranteeing collision-free traversal. This capability rests on three interdependent algorithmic components [6] :

— **Costmap :**

A costmap is a spatial representation that augments the geometric map with traversal cost information. It is constructed by projecting the static map together with real-time sensor readings from cameras and LiDAR onto a uniform grid. Each cell is assigned a scalar cost value that reflects its navigational desirability : low values indicate freely traversable space, whereas elevated values denote proximity to obstacles or regions that are restricted or hazardous.[6]

— **Global Path Planner :**

The global planner operates on the full costmap to compute an optimal, collision-free trajectory from the robot’s current position to the specified goal. It typically employs graph-based search algorithms (e.g., Dijkstra or A\*) or grid-based methods to identify the lowest-cost path through the environment, favouring open space and avoiding high-cost regions.[6]

— **Local Controller (Local Planner) :**

The local controller executes the trajectory produced by the global planner in real time by generating velocity commands that account for both the reference path and dynamic changes in the immediate environment. It continuously selects a local target point on the global path, computes the appropriate steering and velocity adjustments, and issues motor commands at a high frequency, enabling the robot to respond promptly to unforeseen obstacles or deviation.[6]

#### 3.2 Core Components of Navigation Stack

Nav2 decomposes the navigation problem into a set of independent, specialized servers, each responsible for a distinct aspect of the navigation lifecycle. This modular design promotes configurability and facilitates the substitution of individual algorithms without affecting the remainder of the stack. [6]

A detailed description of each component is provided in Table 2.1.

TABLE 2.1 – Nav2 Stack Components [7].

| Component                                       | Description                                                                                                                                                                        |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Adaptive Monte Carlo Localization (AMCL)</b> | A probabilistic localization module that employs a particle filter to continuously estimate the robot’s pose relative to a known map.                                              |
| <b>BT Navigator Server</b>                      | The central execution orchestrator, which uses <b>Behaviour Trees (BT)</b> to coordinate the sequencing of navigation sub-tasks (e.g., global planning followed by local control). |
| <b>Planner Server</b>                           | Maintains the global costmap and computes the optimal collision-free trajectory to the target goal.                                                                                |
| <b>Controller Server</b>                        | Maintains the local costmap and issues real-time velocity commands to track the globally planned path while reacting to dynamic obstacles.                                         |
| <b>Behaviour Server</b>                         | Manages fault-recovery manoeuvres such as in-place rotation or backward motion invoked when the robot becomes immobilized or loses its planned trajectory.                         |
| <b>Smoother Server</b>                          | Post-processes the raw global path to improve its geometric continuity and kinodynamic feasibility prior to execution.                                                             |
| <b>Waypoint Follower</b>                        | Coordinates sequential navigation through an ordered list of intermediate waypoints, delegating each goal to the core navigation pipeline in turn.                                 |
| <b>Velocity Smoother</b>                        | Applies rate limiting and interpolation to the velocity command stream to prevent abrupt accelerations and ensure smooth robot motion.                                             |
| <b>Docking Server</b>                           | Manages the precision manoeuvres required for the robot to autonomously align with and connect to a charging station.                                                              |

Effective navigation presupposes that the robot knows its own position within the map at all times. The following section examines the localization pipeline that the Navigation Stack relies upon.

## 4 Robot Localization

Robot localization is the process by which an autonomous mobile robot continuously infers its pose that is, its position and orientation within the operational environment. It constitutes a prerequisite for every higher-level capability the robot must exhibit : path planning, obstacle avoidance, and mission execution all depend on a reliable and up-to-date answer to the fundamental question “*Where am I ?*”[10]

### 4.1 Wheel Odometry

Odometry is the technique of estimating the robot’s displacement by integrating incremental motion measurements derived from wheel encoder counts. As each encoder pulse corresponds to a known arc length at the wheel circumference, the cumulative count yields an estimate of the total linear distance travelled and, in conjunction with differential measurements between wheels, the change in heading.[8]

### 4.2 Inertial Measurement Unit (IMU)

An Inertial Measurement Unit (IMU) is an electronic sensor that measures the three-dimensional inertial states of the body to which it is attached, capturing both translational and rotational dynamics[9].

It integrates two complementary sensing elements[9] :

- **Gyroscope** : Measures angular velocity (expressed in  $^{\circ}/s$  or  $rad/s$ ) along the three principal axes, enabling the estimation of orientation changes and rotational rates.
- **Accelerometer** : Measures specific force (in  $m/s^2$ ) along each axis, providing information about translational acceleration and, when combined with gravity modelling, about tilt and linear motion.

#### 4.3 Extended Kalman Filter (EKF)

The Extended Kalman Filter (EKF) is a recursive state estimator that represents the robot's pose as a Gaussian distribution parametrized by a mean vector and a covariance matrix encoding estimation uncertainty. Its operation alternates between two complementary stages [10] :

##### 1. Prediction Step (Motion Model) :

Using the robot's kinematic model together with wheel odometry data, the filter projects the current state estimate forward in time to anticipate the robot's next pose. This propagation inevitably increases the covariance, reflecting the accumulation of motion-model uncertainty.[10]

##### 2. Correction Step (Measurement Update) :

External sensor observations, notably IMU orientation readings are incorporated to refine the predicted estimate. The filter computes the Kalman Gain, a weighting coefficient that balances trust between the motion prediction and the sensor measurement according to their respective uncertainties. The resulting corrected estimate exhibits reduced covariance compared with the prediction alone.[10]

#### Key Properties and Integration Role

##### — Fault Tolerance :

By maintaining an explicit model of each sensor's noise characteristics, the EKF can detect and partially discount inconsistent measurements for example, those produced by a slipping wheel, in favour of more reliable concurrent observations from the IMU, thereby improving overall estimate quality[10].

##### — Foundation for Global Localization :

In autonomous navigation pipelines, the EKF is standardly employed to fuse wheel odometry with IMU data, producing a high-frequency, locally accurate pose estimate expressed in the odometry frame. This filtered output subsequently serves as the input motion model for the global localization algorithm, Adaptive Monte Carlo Localization , described in the following section.[10]

#### 4.4 Global Localization : Adaptive Monte Carlo Localization (AMCL)

AMCL is a probabilistic, map-based localization algorithm designed for mobile robots operating in planar environments. It extends the classical Monte Carlo Localization (MCL) framework by introducing an adaptive particle management strategy, implemented via a particle filter, to track the robot's pose against a pre-built occupancy map.[10]

— **Particle Filter Mechanism :**

— **Initialization :**

The algorithm initializes a cloud of weighted particles, each representing a hypothesis about the robot’s current pose. In the absence of any prior pose information, particles are distributed uniformly across the entire map ; otherwise they are seeded around the known initial pose.[10]

— **Update Cycle :**

1. **Prediction :** Each particle is propagated forward according to the robot motion model, incorporating EKF-filtered odometry data to simulate plausible pose transitions.[10]
2. **Correction :** The weight of each particle is updated by computing the likelihood of the actual LiDAR scan given the hypothesized pose, i.e., by comparing the expected sensor reading at that particle’s location with the measurement actually obtained.[10]
3. **Resampling :** Particles are resampled in proportion to their weights : high-likelihood hypotheses are replicated while low-likelihood ones are discarded, causing the particle cloud to concentrate progressively around the robot’s true pose.[10]

— **Adaptive Features (KullbackLeibler Divergence Sampling) :**

The adaptive component of AMCL addresses two practical failure modes [10] :

- **Recovery from Robot Kidnapping :** If the overall particle weight collapses indicating that no hypothesis adequately explains the current sensor observations the algorithm injects a new set of uniformly distributed particles to allow global re-localization.
- **Computational Efficiency :** Once the particle cloud has converged to a compact region, the total number of particles is dynamically reduced, lowering the computational load without sacrificing localization accuracy.

#### 4.5 Synthesis : The Localization Hierarchy

The three components described above operate as a cooperative, layered hierarchy in which each level refines and augments the output of the level below :

1. Wheel odometry and IMU measurements supply raw proprioceptive signals capturing incremental displacement and angular rate.
2. The EKF fuses these signals to generate a smooth, high-frequency pose estimate in the local odometry frame, correcting for the complementary noise characteristics of each sensor.
3. AMCL leverages the EKF output alongside LiDAR scan data to align the robot’s estimated trajectory with the global map, eliminating the long-term drift that the EKF alone cannot suppress.

The localization and navigation mechanisms described above operate on a physical platform whose motion capabilities and kinematic constraints directly condition their performance. This project relies on a differential drive robot, whose operating principles are introduced below.

## 5 Differential Drive Robot

A differential drive robot achieves locomotion through two independently actuated wheels on a common lateral axis, supplemented by passive caster wheels for stability. Heading and velocity are determined entirely by the relative wheel speeds : equal velocities produce straight-line motion, unequal velocities produce a curved path, and opposite velocities produce in-place rotation. This kinematic simplicity, combined with high manoeuvrability in confined spaces, makes it a prevalent choice for indoor mobile robots.[11] Figure 2.2 presents the differential drive behavior.

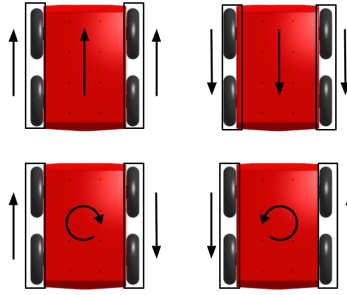


FIGURE 2.2 – Differential drive robot concept[11].

## III Analysis of the Existing Solutions

This section reviews current approaches to multi-task and objective orchestration in autonomous robots, focusing on task sequencing, execution flow control, and system reactivity. It identifies limitations in handling dynamic tasks, conflicting objectives, which motivates the need for a solution aligned with our project goals.

### 1 Classical Decision-Making Architectures in ROS 2-Based Systems

The design of decision-making logic is a central concern in autonomous robotics, as it governs how a robot selects and sequences actions in response to its operational context. Two architectures have been widely employed in ROS 2-based systems for this purpose : Finite State Machines and Behavior Trees.

#### 1.1 Finite State Machines (FSM)

A Finite State Machine (FSM) models a system as a finite collection of discrete states connected by conditional transitions. At any point in time, exactly one state is active, and a transition to a successor state is triggered when its associated condition is met. In robotic applications, each state typically encodes a specific operational mode, such as standby, navigate, or halt making FSMs an intuitive choice for simple, linear task flows[12] .

The following Figure 2.3 presents an example of a finite state machine illustrating state transitions within a robotic task execution process.

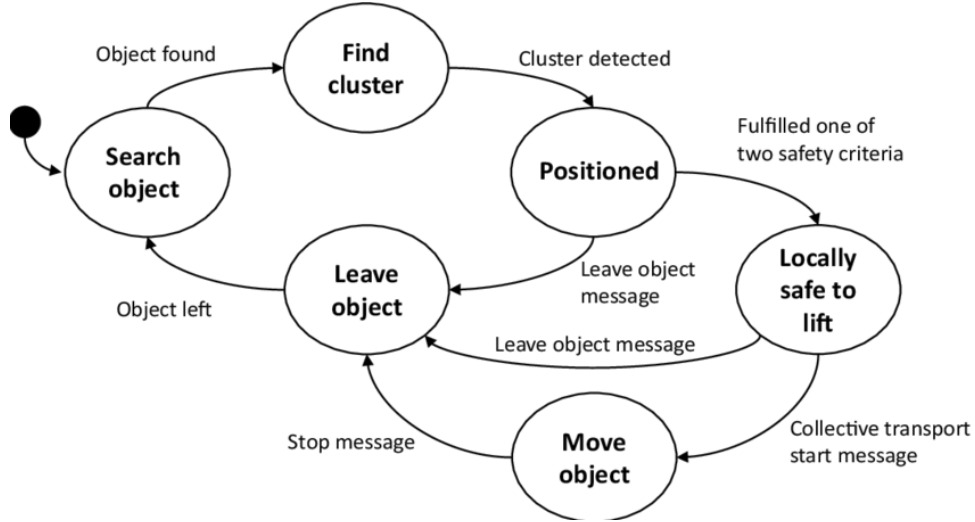


FIGURE 2.3 – Example of a Finite State Machine for robotic task execution[13].

Despite their representational transparency, FSMs do not scale gracefully to complex behavioral requirements. As the number of operational modes grows, the set of transitions that must be explicitly defined expands combinatorially, producing a densely coupled graph that becomes progressively harder to extend and maintain. This phenomenon commonly termed *state explosion* renders FSMs impractical for autonomous systems that must manage a large and evolving set of behaviors[14].

## 1.2 Behavior Trees (BT)

since become a standard execution framework in autonomous robotics. A BT organizes decision logic as a rooted hierarchy of nodes, evaluated through periodic tick signals that propagate from the root downward. Leaf nodes either execute robot actions or evaluate environmental conditions, while internal nodes apply control-flow policies, such as sequential or alternative execution based on the Success or Failure status returned by their children[15].

To illustrate the hierarchical organization of Behavior Trees, the following Figure 2.4 presents an example of a typical BT structure.

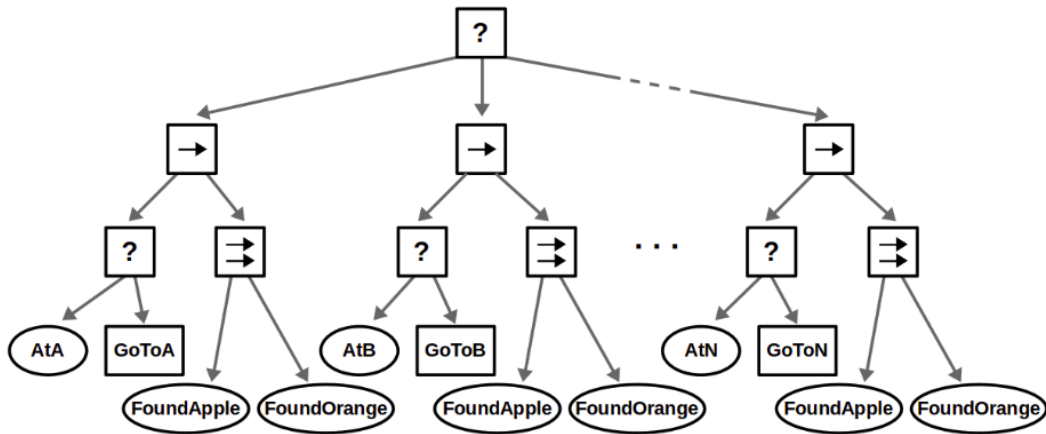


FIGURE 2.4 – Example structure of a Behavior Tree [15].

The principal strength of BTs lies in their modularity and reactivity : subtrees can be developed and reused independently, and because conditions are re-evaluated on every tick, higher-priority behaviors can preempt ongoing ones without requiring explicit inter-state transitions. To support behaviors that require persistent context, modern BT implementations supplement the tree with a blackboard a shared data store through which nodes exchange and retain information across tick cycles [15] .

### 1.3 Comparative Analysis

To highlight the fundamental differences between classical decision making architectures the Table 2.2 compares Finite State Machines and Behavior Trees in terms of reactivity, modularity, readability, and design suitability in robotic systems.

TABLE 2.2 – Comparison between Finite State Machines (FSMs) and Behavior Trees (BTs)

| Criterion          | Finite State Machines (FSM)                                                 | Behavior Trees (BT)                                                           |
|--------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Reactivity         | Static, transitions are predefined and require manual updates when scaling. | Dynamic, evaluated at every tick, allowing immediate reaction and preemption. |
| Modularity         | Low, states are tightly coupled and changes propagate across transitions.   | High, subtrees are independent and reusable components.                       |
| Readability        | Good for small systems but becomes hard to maintain as complexity grows.    | High, hierarchical structure remains clear even in complex systems.           |
| Design Suitability | Best for simple and stable systems with predictable behavior.               | Best for complex autonomous systems requiring flexibility and concurrency.    |

## IV Limitations and Critiques of the Existing Solutions

Existing solutions present several shortcomings and issues that still need to be addressed. Despite their advancements, they continue to exhibit important limitations, which will be outlined as follows :

— **Limited spatial and geometric reasoning :**

Both Finite State Machines and standard Behavior Trees operate on discrete, symbolic representations of the world. They respond to predefined conditions and trigger actions accordingly, but do not reason about the geometry of the environment. This absence of spatial understanding confines the robot to reactive, surface-level behavior rather than genuine situational awareness.

— **Lack of adaptive autonomy :**

Existing architectures assume a relatively stable task environment. Execution follows predetermined logic with limited capacity for runtime adaptation the robot cannot autonomously reorder its task queue. This rigidity is problematic in real-world deployments, where conditions often deviate from design-time assumptions.

— **No mechanism for objective evaluation :**

Current systems provide no reasoning about conflicts between competing objectives. The robot cannot evaluate which objective to prioritise, under what conditions to accept a new assignment.

— **Absence of experiential memory :**

Neither architecture retains past observations or recalls previous environmental conditions.

Taken together, these limitations reveal a clear architectural gap between what current solutions offer and what the present project genuinely requires, a gap that the proposed solution, presented in the following section, is specifically designed to address.

## V Proposed Solution

The proposed Solution allows a robot to evaluate multiple objectives, compare between them based on risk, context, and cost, and execute selected tasks safely in real time. By combining a Vision-Language Model (VLM) for perception, a Large Language Model for natural language parsing, and a deterministic decision engine for mission-level evaluation , with Behavior Tree based task execution (Behavior Trees as execution backbone) within a ROS 2 infrastructure.

This solution achieves a clear separation between strategic decision making and reactive control. This hybrid approach enables robots to continuously adapt their behavior to changing environmental conditions, mission priorities, and system constraints, while maintaining robustness and safety during execution.

### 1 Three Layer Architecture

Our architecture is organized into three distinct yet tightly interconnected layers, each contributing an essential function in the robots operational loop. it enable the robot to reason, assess, and act in complex environments. By combining structured execution, decision-making, and spatio-temporal memory, the system is able to interpret user objectives, evaluate tasks , and adapt behavior to dynamic conditions.

#### 1.1 Execution Layer

This layer translates high-level directives into concrete robotic behaviors. Through Behavior Trees integrated with ROS 2, the robot can act reliably in real time, react to changing conditions, and maintain robustness during navigation and task execution.

#### 1.2 Decision Layer

This layer forms the core of the robot’s decision-making. It is responsible for mission evaluation and orchestration, determining what the robot should do and when it should do it. The system assesses mission objectives ,battery autonomy, ongoing tasks, risk zones, and operational costs then decides whether to accept, counter, or reject new assignments.

By interpreting user instructions through Large Language Model based(LLM) parsing and applying rule based decision logic, this layer connects human intent to operational constraints.

### 1.3 Memory Layer

The Memory Layer provides the system with spatio-temporal memory, enabling it to build and use past experiences. By storing temporal data, spatial positions, and scene observations captured by a Vision-Language Model (VLM), the robot gains the ability to integrate previous observations into current decision-making. This memory strengthens the evaluation process by recalling relevant past events. As a result, the robot evolves from purely reactive behavior to context-aware, experience-informed intelligence.

## VI Conclusion

In this chapter, the necessary foundations have been established, followed by an analysis of existing solutions. This analysis has revealed several challenges that require further consideration. These observations have guided the identification of key requirements to be addressed. The remainder of this report presents the proposed solution and specifies the requirements that govern its implementation.

# Requirements Specifications

## I Introduction

Having established a thorough understanding of the domain through the literature review and contextual analysis presented in the previous chapter, we now proceed to formally specify the requirements governing the development of the multi-task orchestration project. This chapter identifies the actors interacting with the system, defines both functional and non-functional requirements, and presents the use cases that model the intended interactions between the system and its environment. These specifications constitute the contractual foundation upon which the design and implementation phases, developed in the subsequent chapters, are built.

## II Requirements Analysis

Establishing a clear and structured set of requirements is an essential step in any rigorous software development process. The requirements governing this project are organized into two complementary categories : functional and non-functional requirements.

### 1 Functional Requirements

Functional requirements define the specific behaviors and capabilities that the system must provide. The following table 3.1 enumerates the functional requirements of the system.

TABLE 3.1 – Functional Requirements

| ID    | Requirement                           | Description                                                                                                                                                                                                                                          |
|-------|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-01 | Human Operator Mission Input          | The system shall accept single or multi-objective mission assignments submitted by the user through a web interface.                                                                                                                                 |
| FR-02 | Mission Parsing and Decision          | The system shall parse operator instructions through a LLM that extracts intent and action type. followed by A rule-based decision engine that evaluates feasibility.                                                                                |
| FR-03 | Autonomous Navigation                 | The system shall execute navigation tasks through the ROS 2 Navigation stack, including global path planning, local obstacle avoidance, and real-time velocity control.                                                                              |
| FR-04 | Multi-Action Chain Sequencing         | The system shall detect and sequence multi-step chain missions from a single operator message.                                                                                                                                                       |
| FR-05 | Battery-Aware Constraint Evaluation   | Before accepting a task, the system simulates energy consumption by calculating the total travel distance, and predicting the final battery level remaining after all queued tasks are completed.                                                    |
| FR-06 | Task assess and Counter-Proposal      | The system shall decide task acceptance by returning accept, decline, or counter decisions. Counter-proposals shall include a dock-then-execute alternative and automatically re-queue the task after charging.                                      |
| FR-07 | High-Priority Task Preemption         | Upon acceptance of a high-priority mission, all running and queued tasks shall be interrupted, and the remaining task queue shall be revalidated according to nearest-first proximity order following the completion of the high-priority objective. |
| FR-08 | On-the-Way Interrupt and Resume       | The system shall detect when a newly accepted task destination is closer than the current task. In such cases, the current task shall be preempted, the closer task executed first, and the original task resumed automatically afterward.           |
| FR-09 | Behavior Tree Execution Feedback Loop | The system shall receive structured execution events from the Behavior Tree runner and process success, failure, and cancellation events to update task status, recompute costs, and dispatch the next task.                                         |

| ID    | Requirement                        | Description                                                                                                                                                                                                                             |
|-------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-10 | Manual Task Cancellation           | The system shall allow the user to cancel any task (running, queued, or interrupted) via the mission control panel, triggering immediate cursor recomputation and next-task dispatch.                                                   |
| FR-11 | Autonomous Docking and Recovery    | The system shall autonomously trigger docking when battery falls below a safety threshold. The robot shall detect the docking tag, align, charge, and execute recovery behaviors in case of docking failure (retry mechanism).          |
| FR-12 | Spatio-Temporal Memory Integration | The system shall build a spatio-temporal memory storing past observations, forming a memory of recent environmental events for decision-making. It shall query this memory to enrich mission evaluation.                                |
| FR-13 | Real-Time Status Panel             | The system shall provide real-time execution feedback via a web interface, including current battery level, predicted post-queue battery, task queue with status and cost, robot position, and an activity log with timestamped events. |

## 2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that govern how the system must behave, rather than what it must.

The following table 3.2 enumerates the Non-functional requirements of the system.

TABLE 3.2 – Non-Functional Requirements of the Proposed System

| ID     | Category              | Description                                                                                                                                                          |
|--------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Real-Time Performance | The Decision Layer must produce a task scheduling decision within an operationally acceptable latency under normal conditions so as not to delay mission execution.  |
| NFR-02 | Reliability           | The system must continue operating correctly in the presence of sensor noise, wheel slippage, and transient ROS 2 communication disruptions without mission failure. |
| NFR-03 | Safety                | The robot must never execute a task if the battery level is insufficient.                                                                                            |
| NFR-04 | Memory Persistence    | Memory records must persist across robot sessions and remain accessible after system restart, allowing past mission observations to support future decision-making.  |

### III Requirement Specification

This section formalizes the system requirements through a use case model, identifying the interacting actors and describing their roles and interactions with the system.

#### 1 Actors Identification

In this project, three key actors have been identified :

- **User** is the primary human actor. He interacts with the system through the mission control interface by submitting mission requests, tracking their execution status, and receiving system responses.
- **Robot** is the primary system actor. It executes the missions accepted and reports its operational status back to the system.
- **Database** is the secondary system actor. It ensures the persistence of past observation records and responds to queries issued by the system during mission evaluation.

#### 2 Global Use Case Diagram

The global use case diagram provides a high-level overview of all interactions between the identified actors and the system. It groups the principal use cases within the system boundary and establishes the inclusion and extension relationships between them.

The following Figure 3.1 illustrates the overall functional organization of the proposed orchestration system and the interactions between its actors and core services.

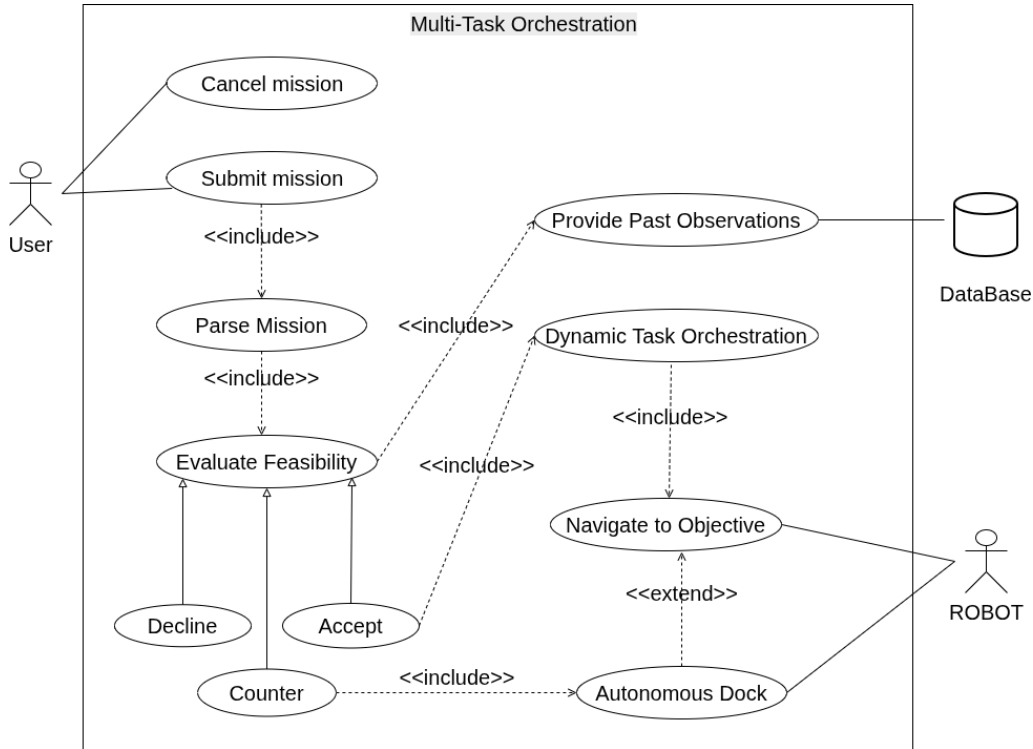


FIGURE 3.1 – Global Use Case Diagram.

The diagram highlights three main interaction flows. The User initiates missions and cancellations through the interface, triggering the decision cycle within the system. The Robot receives dispatched navigation goals and reports execution feedback. the Database is queried during mission evaluation to enhance decision-making.

To further clarify, the following table 3.3 provides a detailed description of each use case presented in the diagram.

TABLE 3.3 – Global Use Case Descriptions

| Use Case                   | Description                                                                                                                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Submit Mission             | The User submits a single or multi-objective mission through the web interface.                                                                                                                                      |
| Cancel Mission             | The User cancels any active, queued, or interrupted task. The system aborts the execution, updates the task queue.                                                                                                   |
| Parse Mission              | The system parses the natural language instruction, extracts the structured intent, and forwards it to the decision engine.                                                                                          |
| Provide Past Observations  | The system queries the database to retrieve past spatial observations.                                                                                                                                               |
| Evaluate Feasibility       | The system evaluates the mission against battery constraints, queue conflicts, and retrieved risk context, returning an accept, decline, or counter verdict to the User.                                             |
| Dynamic Task Orchestration | Upon acceptance, the system integrates the mission into the active queue, applies priority preemption and proximity-based reordering and on-the-way interrupt, and dispatches the next executable goal to the Robot. |
| Navigate to Objective      | The Robot executes the dispatched navigation goal, performing path planning and obstacle avoidance, then reports execution status to the Decision Layer upon arrival.                                                |
| Autonomous Dock            | the Robot autonomously triggers the docking sequence.                                                                                                                                                                |

### 3 Use Cases Description

The following subsections provide detailed textual descriptions of the four most representative use cases of the system, selected as they directly reflect the core contributions of the project : mission-level analysis, orchestration, and execution.

#### 3.1 Use Case : Submit Mission

Table 3.4 provides the textual description of the use case Submit Mission.

TABLE 3.4 – Textual Description of UC-01 : Submit Mission

| Field            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-01                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Title            | Submit Mission                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Primary Actor    | User                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Objective        | Allow the user to submit a single or multi-objective mission through the web interface and produce a structured, executable task sequence ready for feasibility evaluation.                                                                                                                                                                                                                                                                                              |
| Preconditions    | The web interface is running.<br>The LLM based parsing module is available and responsive.                                                                                                                                                                                                                                                                                                                                                                               |
| Main Flow        | <ol style="list-style-type: none"><li>1. The user enters a natural language mission instruction in the web interface and submits it.</li><li>2. LLM extracts the intent, action type, and target locations from the instruction.</li><li>3. For multi-step instructions, the system detects and sequences the actions using verb inheritance.</li><li>4. The structured mission data is transmitted to the decision engine to initiate feasibility assessment.</li></ol> |
| Alternative Flow | Ambiguous Command : If the LLM is unable to extract a coherent intent, the system returns a message to the user .                                                                                                                                                                                                                                                                                                                                                        |
| Postconditions   | A structured and sequenced mission intent has been produced and forwarded to the decision engine.                                                                                                                                                                                                                                                                                                                                                                        |

### 3.2 Use Case : Evaluate Feasibility

Table 3.5 provides the textual description of the use case Evaluate Feasibility.

TABLE 3.5 – Textual Description of UC-02 : Negotiate Feasibility

| Field             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID       | UC-02                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Title             | Evaluate Feasibility                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Primary Actor     | User                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Secondary Actors  | Robot, Database                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Objective         | To evaluate mission feasibility and provide an accept, decline, or counter verdict.                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Preconditions     | A structured mission intent has been successfully produced by the submission stage .                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Main Flow         | <ol style="list-style-type: none"><li>1. The system queries the Database to retrieve past risk observations.</li><li>2. The system predict the total energy cost of the new mission.</li><li>3. The system evaluates the mission against battery budget ,and retrieved risk context.</li><li>4. If constraints are met, the system returns an Accept verdict and forwards the mission to the Orchestration Engine.</li><li>5. The user receives a confirmation message through the web interface.</li></ol> |
| Alternative Flows | <p><b>Counter :</b> if battery is insufficient to complete the full mission but sufficient after docking, the system proposes a dock-then-execute counter-proposal. The task is automatically re-queued after charging.</p> <p><b>Decline :</b> the system returns a Decline decision with a justification message to the user.</p>                                                                                                                                                                         |
| Postconditions    | The user has received a clear accept, counter, or decline response.<br>If accepted, the mission is forwarded to the dynamic orchestration for queue integration.                                                                                                                                                                                                                                                                                                                                            |

### 3.3 Use Case : Dynamic Task Orchestration

Table 3.6 provides the textual description of the use case Dynamic Task Orchestration.

TABLE 3.6 – Textual Description of UC-03 : Dynamic Task Orchestration

| Field            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-03                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Title            | Dynamic Task Orchestration                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Primary Actor    | System                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Secondary Actors | User                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Objective        | dynamically manage the mission queue.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Preconditions    | A mission has received an Accept verdict from the decision engine.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Main Flow        | <ol style="list-style-type: none"><li>1. Priority Evaluation : The system identifies the priority level of the incoming task. High-priority missions trigger an immediate termination signal to the currently running task to clear the execution pipeline.</li><li>2. Task Preemption : The system interrupts the active mission, persists its state for future resumption, and promotes the high-priority mission to the head of the execution queue .</li><li>3. Proximity-Based Optimization : For standard tasks, the system re-orders the queue using a nearest-first proximity algorithm to minimize total travel distance and reduce energy overhead .</li><li>4. Status Dispatch : The system pushes the updated task sequence and the re-calculated predicted battery levels to the User via the web interface.</li><li>5. Execution Loop : The prioritized goal is dispatched to the Robot's execution layer to begin physical movement .</li></ol> |
| Alternative Flow | On-the-Way Interrupt : If a new mission destination is closer than the current target, the system preempts the current task to execute the closer objective first.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Postconditions   | An optimized task sequence is active, and the Robot has begun execution of the highest-priority goal.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

### 3.4 Use Case : Navigate to Objective

Table 3.7 provides the textual description of the use case Navigate to Objective.

TABLE 3.7 – Textual Description of UC-04 : Navigate to Objective

| Field            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-04                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Title            | Navigate to Objective                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Primary Actor    | Robot                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Objective        | To physically move the robot to a specific target coordinate while ensuring obstacle avoidance and providing real-time progress updates.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Preconditions    | <p>A valid navigation goal has been formally approved and dispatched by the Decision engine.</p> <p>A traversable path exists between the current robot position and the target mission location.</p> <p>The robot has a valid estimate of its position within the operational environment.</p>                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Main Flow        | <ol style="list-style-type: none"><li>1. Goal Acquisition : The robot's execution layer receives the target coordinates and initiates the movement sequence.</li><li>2. The robot generates continuous steering and speed commands to follow the path while dynamically reacting to unforeseen obstacles.</li><li>3. Arrival Confirmation : Once the robot successfully reaches the destination coordinates, it issues an execution feedback signal, allowing the system to verify the task as completed.</li><li>4. System Synchronization : The system marks the task as finished, updates the predicted battery levels based on real consumption, and prepares to trigger the next task in the sequence.</li></ol> |
| Alternative Flow | If the robot fails to reach the destination, the mission is aborted. The system then re-evaluates the remaining task queue and notifies the operator.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Postconditions   | <p>The robot has arrived at the target location or has formally reported a mission abortion.</p> <p>The user interface reflects the updated mission status.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

## IV Conclusion

This chapter defined the requirements for the multi-task orchestration system. It identified the system actors, specified functional and non-functional requirements, and clarified interactions through the use case model. These specifications serve as the contractual reference for the architectural design presented in the following chapter.

# Chapter 4

## Project Design

### I Introduction

In this chapter, we present the design phase of our system, which represents a key step in the development process. The objective of this phase is to translate the requirements identified in the previous chapter into a clear and organized system architecture. We begin by introducing an overview of our solution and its different architectural levels. Then, we detail the system design using various Unified Modeling Language (UML) diagrams in order to illustrate the structure of the system, the interactions between its components, and the behavior governing its operation.

### II Architectural Design

This section presents the architectural design of the proposed system and the different mechanisms involved in its operation. The objective of this design is to provide a modular and organized structure capable of supporting mission assess, dynamic orchestration, and autonomous robotic execution.

#### 1 Global Architecture

The proposed architecture follows a layered approach composed of three tightly coupled levels, each responsible for a specific set of cognitive and operational functionalities based on a remember ,think ,act paradigm. The three layers are :

- **Memory Layer**

This layer continuously captures environmental observations through a VLM-based scene perception component, stores them as spatio-temporal records, and is queried by the Decision Layer at mission evaluation time, returning relevant historical observations that enrich situational awareness. It embodies the remember stage.

### — Decision Layer

Constitutes the core of the system. It receives natural language mission instructions from the human operator processes them through a two-stage pipeline :first parsing mission intent then evaluates their feasibility against operational constraints (battery budget,risk context), and orchestrates the resulting task queue through dynamic prioritization and adaptive reordering mechanisms. It embodies the think stage.

### — Execution Layer

Translates high-level navigation directives into safe, real-time robotic behaviors through a Behavior Tree execution engine that governs task sequencing, monitors progress, and triggers recovery on failure. It ensures that decisions produced by the Decision Layer are physically executed through autonomous navigation and continuous obstacle avoidance. It embodies the act stage.

Therefore , The following figure 4.1 illustrates the global architecture of the proposed system, presenting the three layers and the data flows connecting them.

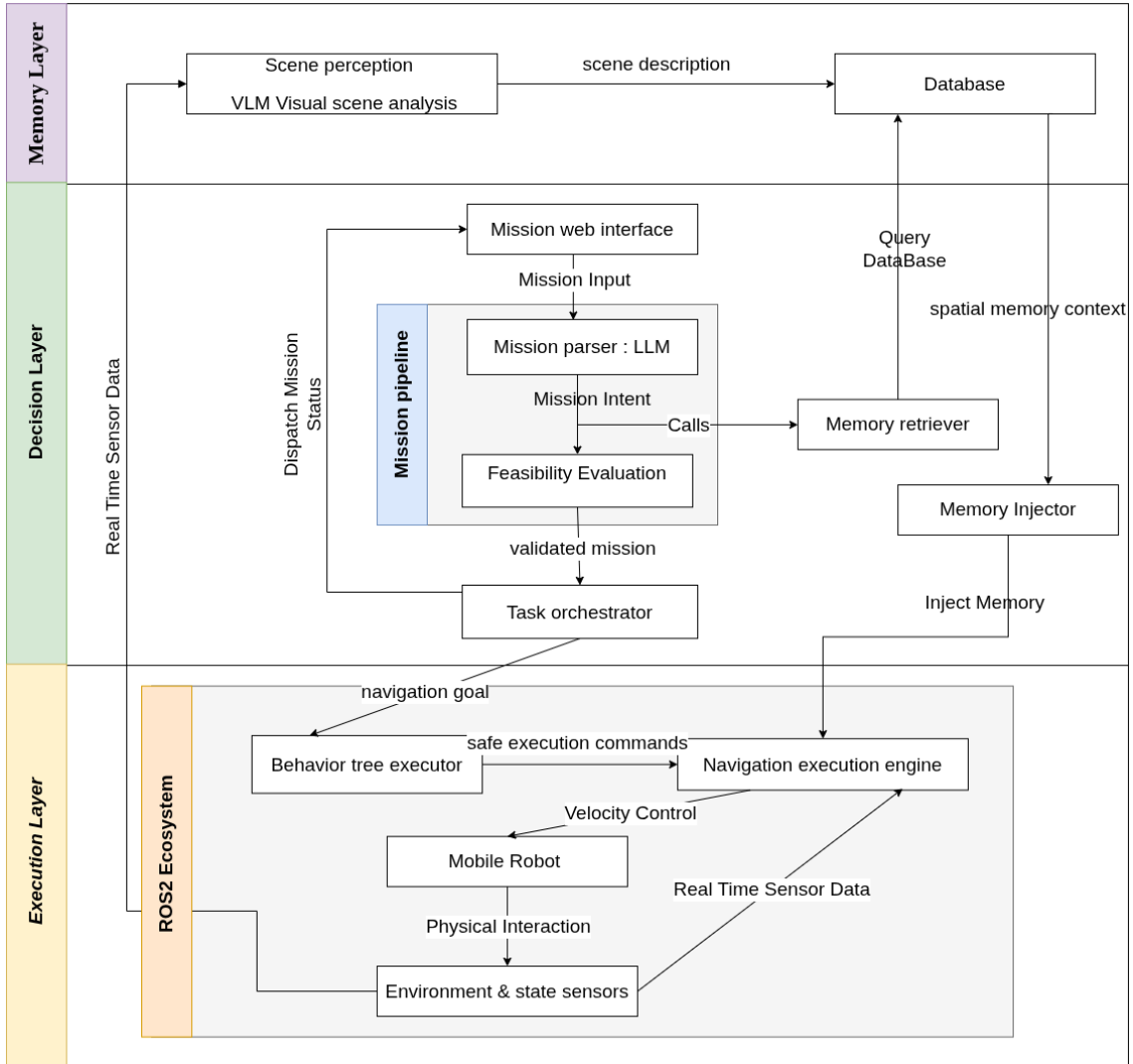


FIGURE 4.1 – Global System Architecture Diagram.

### III Detailed Architecture

This section presents the detailed design of the proposed system. It first introduces the core operational logic and behavioral rules governing the robots decision-making. It then proceeds through a set of Unified Modeling Language( UML) diagrams describing both the structural and behavioral aspects of the system. Together, these diagrams provide a clear overview of the system architecture, component relationships, and execution flow.

#### 1 Conceptual Design Foundations

Before presenting the structural and behavioral diagrams of the proposed system, it is necessary to establish the conceptual models upon which the entire Decision Layer is built. The class diagram and sequence diagrams that follow can only be fully understood in light of the design choices presented in this section. Four core models govern the behavior of the orchestration engine : the task modeling , the energy study and battery cursor model, the dynamic preemption model, and the episodic memory retrieval model.

These models are assumed as the initial conceptual basis of the proposed system, serving its intended purpose, while remaining open to future extensions.

##### 1.1 Task and Priority Model

The system operates over a predefined set of task types, each corresponding to a distinct category of robotic intervention. Two principal task types were identified and formally modeled to provide sufficient behavioral coverage for evaluating and demonstrating the orchestration logic.

- **Delivery (High Priority) :**

Delivery tasks require the robot to transport an item from a source location to a target location. These tasks are classified as high-priority due to their time-sensitive nature . Upon acceptance of a delivery task, all currently running and queued tasks are interrupted and suspended, and the delivery is promoted to the head of the execution pipeline.

- **Go To (Standard Priority) :**

Navigation tasks require the robot to move to a specified location without performing any secondary action upon arrival. go to tasks are classified as standard priority and participate in the queue's proximity optimization logic.

In addition to single action tasks, the system supports multi-step chain missions, in which a single instruction triggers an ordered sequence of tasks. A chain mission is treated as a composition of individual tasks evaluated sequentially. For chains has same action type the system applies proximity-based reordering to minimize total travel distance. For chains submitted under low battery conditions , the system reorders tasks by ascending energy cost, prioritizing the least expensive steps to maximize the number of tasks that can be completed before a mandatory docking event.

## 1.2 Predictive Energy Forecasting

The second design decision addressed how to determine whether the robot has sufficient energy to accept a new task, not only in terms of its current battery level, but in terms of its projected battery after all previously accepted tasks have been executed. To address this, a predictive battery cursor model was designed.

- **Energy Cost Estimation :**

The energy cost of any task is modeled as a function of the distance the robot must travel to complete it.

- **The Battery Cursor :**

The battery cursor projects the estimated battery level after executing all queued tasks. It is initialized to the actual battery when the queue is empty and decremented by each accepted task's cost. Feasibility is evaluated against the cursor, not current battery, preventing double-counting of resources.

The cursor is rebuilt from scratch after every queue change (add, remove, reorder, complete). Each task's cost is recomputed from its correct start position, eliminating accumulated estimation errors.

## 1.3 Dynamic Preemption Model

The third design decision addressed the question of how the system should behave when a newly accepted task must take precedence over tasks already in progress. Two distinct preemption scenarios were identified, each requiring a dedicated mechanism.

Therefore, Table 4.1 below summarizes and contrasts the two preemption mechanisms.

TABLE 4.1 – Preemption Mechanisms

| criteria           | Delivery Preemption                 | On-the-Way Interrupt                       |
|--------------------|-------------------------------------|--------------------------------------------|
| Trigger            | Delivery task accepted              | New destination closer than current target |
| Interruption Scope | All running and queued tasks        | Only current running task                  |
| Resumption Order   | Nearest-first from dropoff location | –                                          |
| Cost Recalculation | All resumed tasks                   | Suspended task only                        |

## 1.4 Memory Integration

The fourth design decision concerned how the proposed solution should leverage past observations to improve current mission evaluation. the Memory Layer was designed to supply the Decision Layer with spatial context before each task is dispatched. For this purpose, two types of observations were retained :

- paths entirely blocked by obstacles
- paths identified as heavily crowded.

## 2 Structural Aspect

This section delineates the static architecture of the proposed system through the class diagram.

The Figure 4.2 details the primary classes, their constituent attributes and methods, and the structural relationships established between them.

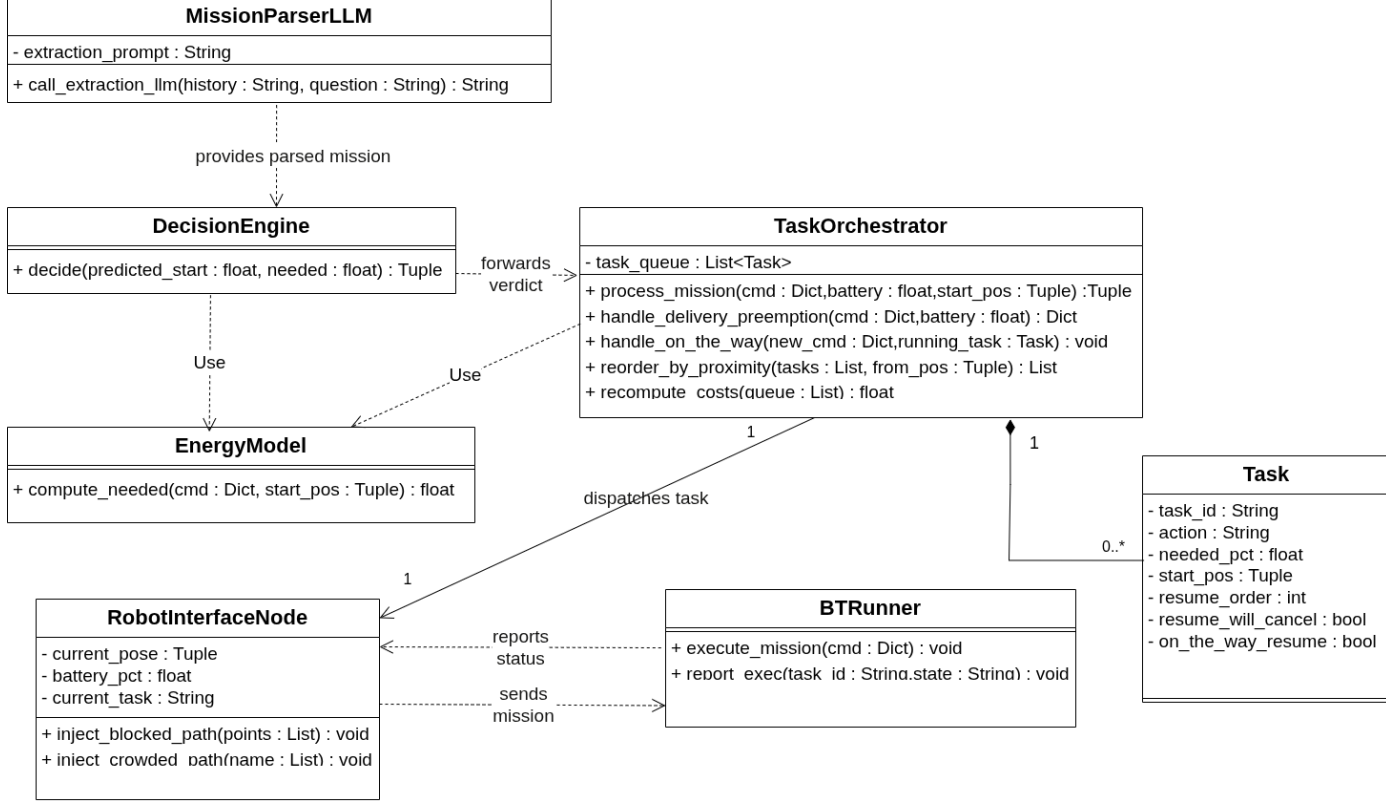


FIGURE 4.2 – Class diagram

## 3 Behavioral Aspect

This part describes the dynamic behavior of the system during execution through sequence diagrams representing the interactions and information exchange between the different system components. starting with the global nominal execution flow and then moving to specific situations.

### 3.1 General Sequence Diagram

The general sequence diagram , shown in Figure 4.3, illustrates the nominal flow interactions between the different components of the system.

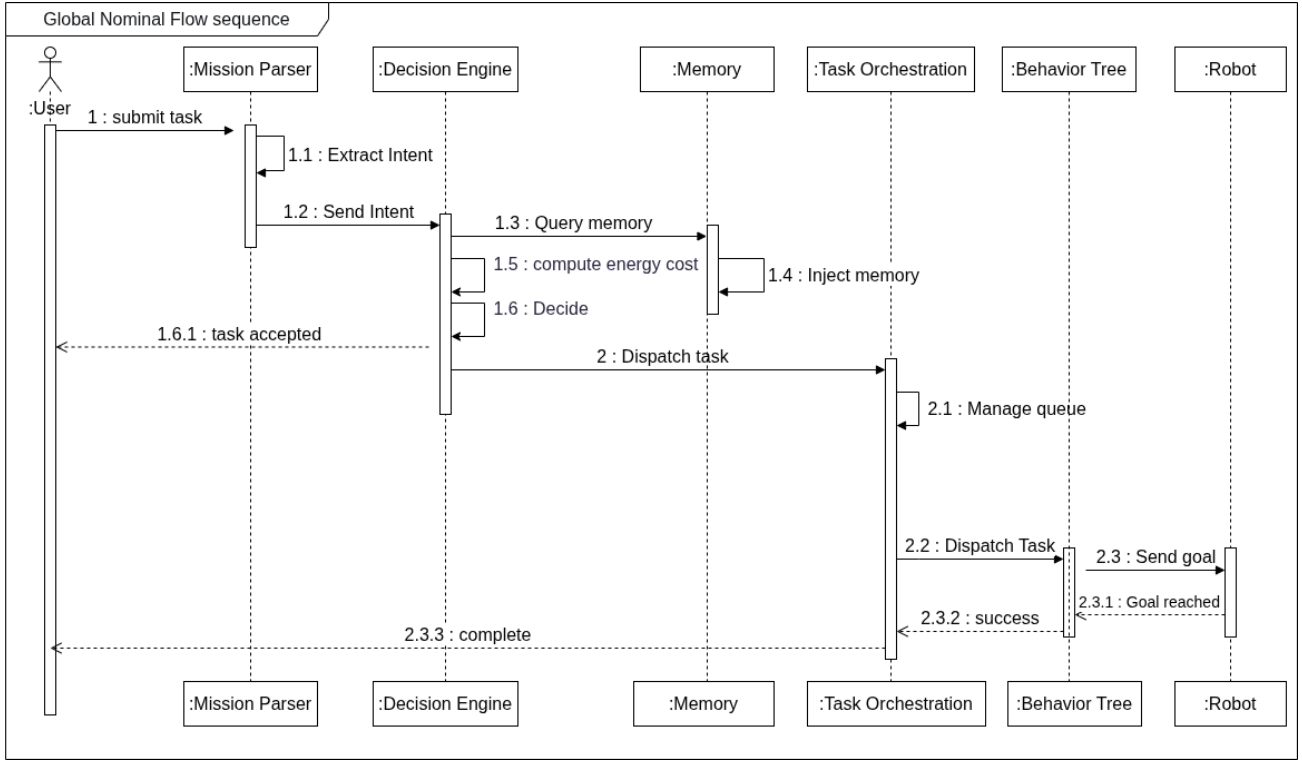


FIGURE 4.3 – Nominal Flow Sequence Diagram

### 3.2 Memory injection sequence

After detailing the nominal mission lifecycle, we now examine the core of the system : how memory influences feasibility evaluation.

The sequence diagram below presented by figure 4.4 shows the complete flow from memory retrieval and injection to path aware energy computation and final decision

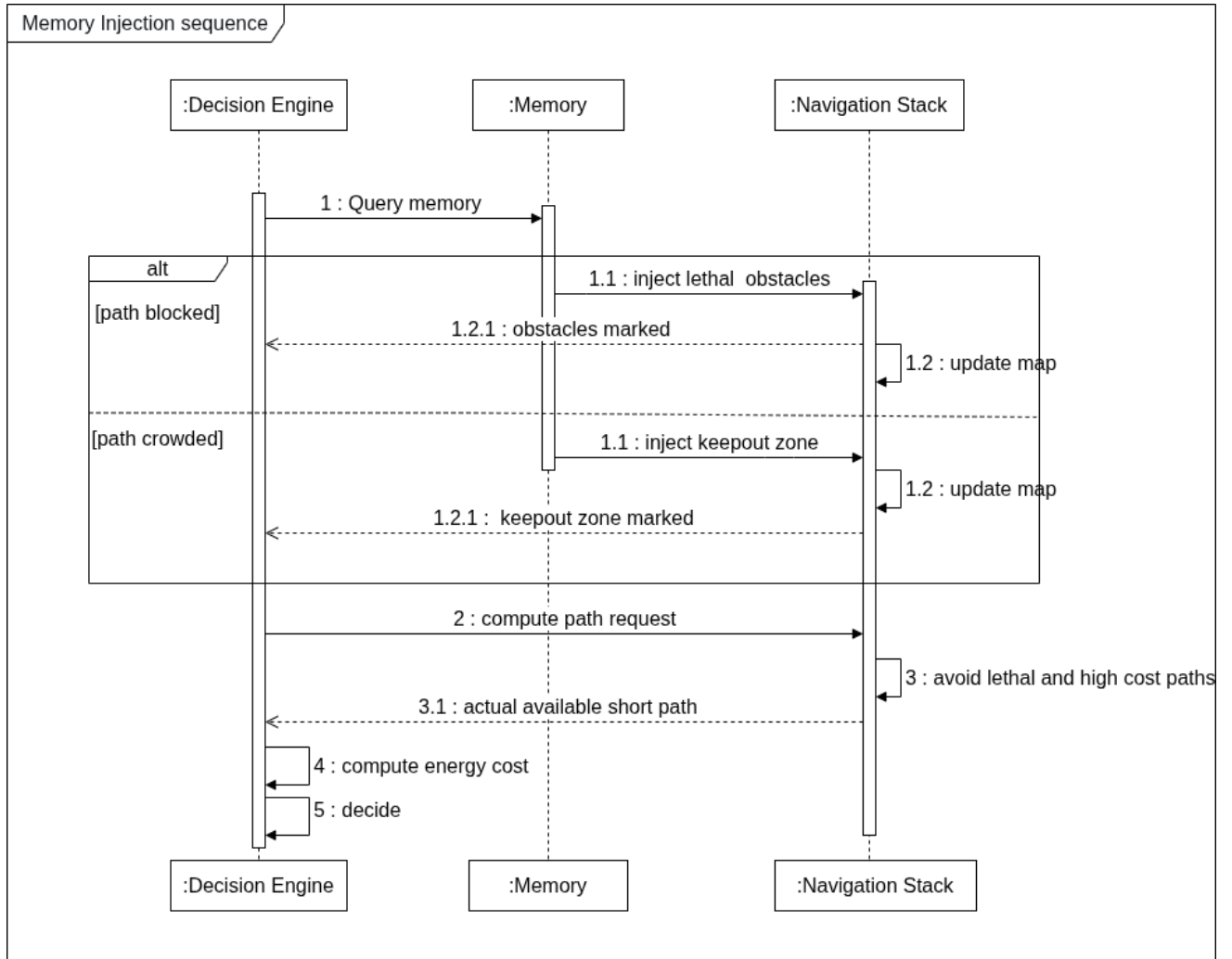


FIGURE 4.4 – Memory Injection Sequence Diagram

### 3.3 Feasibility Evaluation sequence

Having seen how memory shapes the path and energy estimate, we now turn to the decision logic itself.

The Figure 4.5 presents the three verdicts Accept, Counter, and Decline

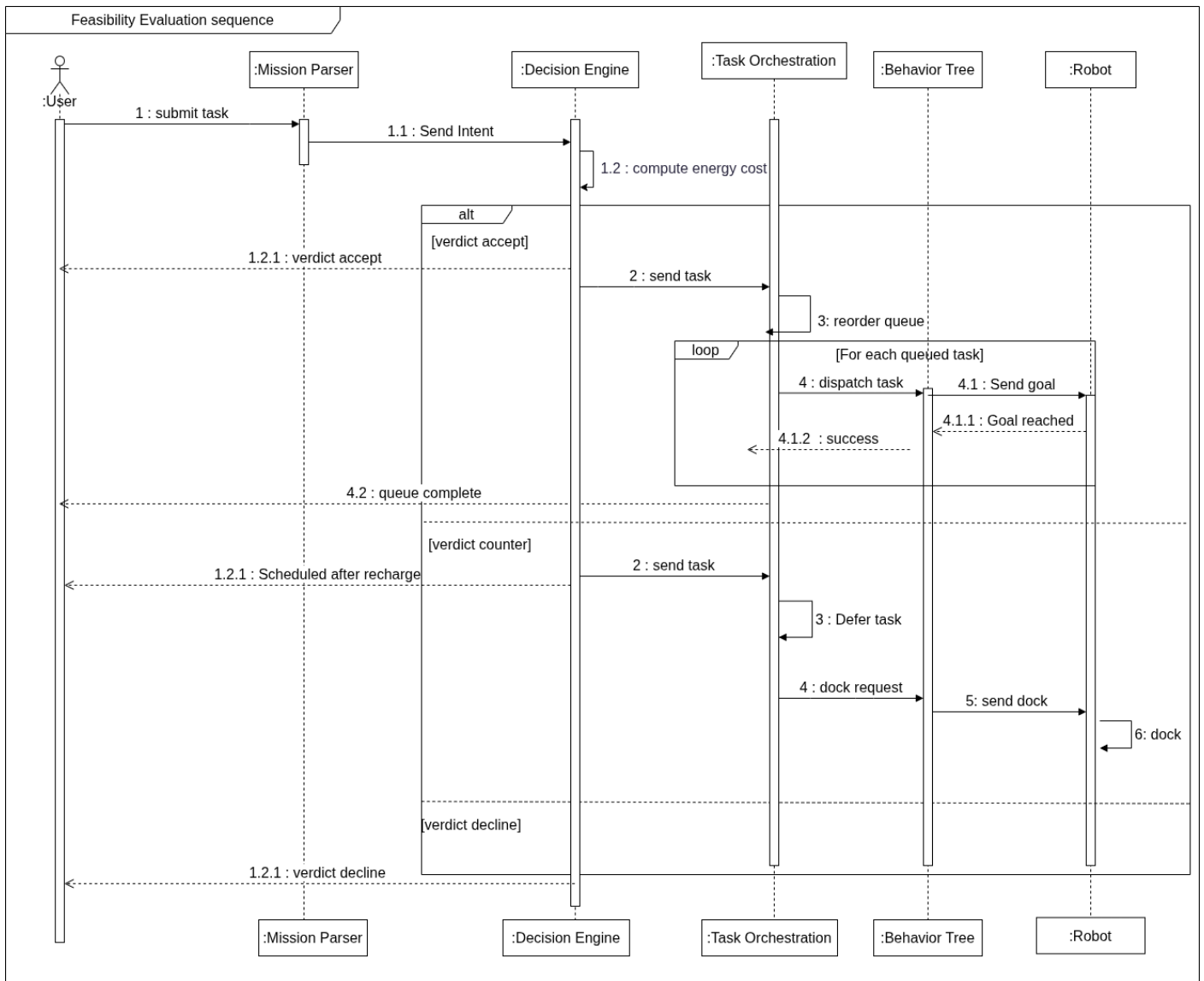


FIGURE 4.5 – Feasibility Evaluation sequence Diagram

### 3.4 Delivery Preemption sequence diagram

The previous diagrams considered standard task execution. The following sequence diagram in Figure 4.6 illustrates how the system handles high-priority tasks that preempt all ongoing missions.

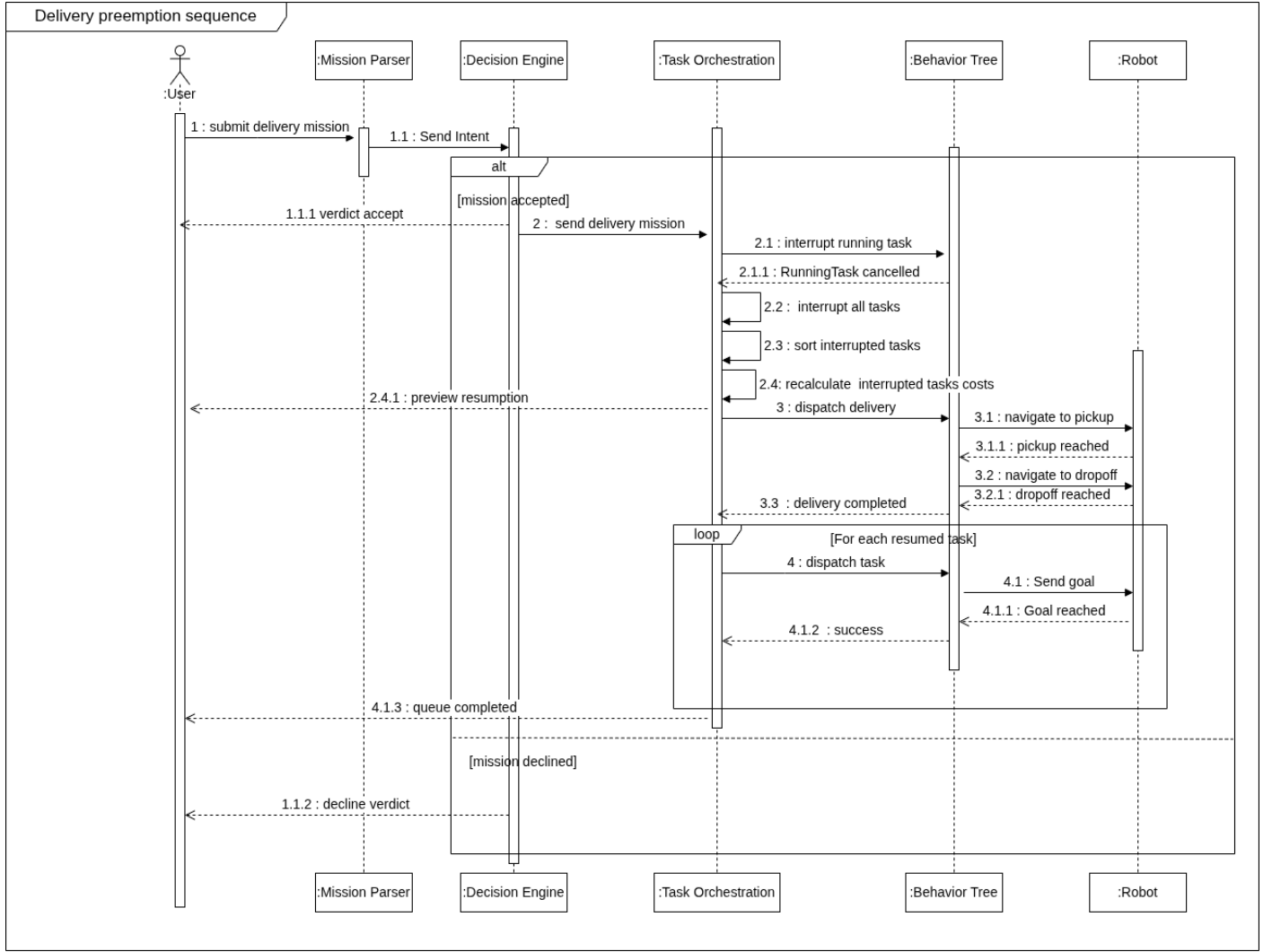


FIGURE 4.6 – Delivery Preemption sequence Diagram

## IV Conclusion

This chapter provided an architectural overview of the proposed solution, mapping its structural dependencies and functional workflows through class and sequence diagrams. This foundation leads directly to the implementation phase detailed in the following chapter.

# Project Realisation

## I Introduction

This chapter focuses on the practical implementation of the system. We first describe the development environment and the technologies used. Then, we explain the work completed during the project. Finally, we present the performance results of the solution.

## II Development environment

This section outlines the development setup and the tools used during the implementation of the project. It provides details on the technologies and infrastructure that supported the development process.

### 1 Hardware Configuration

To ensure the system performed correctly during development and testing, Table 5.1 details the hardware specifications that were used.

TABLE 5.1 – Hardware Specifications of the Development Environment

| Component        | Specification                        |
|------------------|--------------------------------------|
| Processor (CPU)  | 13th Gen Intel(R) Core(TM) i7-13620H |
| Memory (RAM)     | 16 GB                                |
| Graphics (GPU)   | NVIDIA GeForce RTX 4050 (6 GB VRAM)  |
| Storage          | 300 GB SSD                           |
| Operating System | Ubuntu 24.04.4 LTS                   |

## 2 Technology Choices

The proposed system is built upon a set of complementary technologies that work together to realize its functional requirements. This section presents the technical stack, detailing the role of each component and the justification for its selection.

### 2.1 Robot Operating System (ROS2) Jazzy

ROS 2 Jazzy was adopted as the central middleware of the proposed robotic architecture, managing communication between all distributed modules across the implemented system through a topic-based and action-based communication model. It was chosen because it is the industry standard for robot development, widely tested, and comes with a complete navigation stack (Nav2) that integrates seamlessly with the rest of the system.

### 2.2 Nav2

Nav2 was used as the primary navigation stack to handle all robot movement, coordinating a set of modular software plugins that work together to plan the shortest and safest path from the robot's current position to a target location. Specifically, it utilizes Behavior Trees to manage complex decision-making, allowing the system to continuously avoid obstacles detected by sensors while dynamically recalculating routes in real-time. It was selected for its native integration with the ROS 2 ecosystem that allowed us to focus on higher-level orchestration and memory logic without having to build low-level motion control from scratch.

### 2.3 Behavior Tree

Behavior Trees (BT) are used as the execution backbone of the system. A Behavior Tree is a hierarchical structure of nodes that controls how the robot sequences tasks, responds to failures, and handles interruptions. BTs were adopted because they are the standard ROS 2 pattern for task coordination, offering modularity, reusability, and built-in support for preemption and recovery behaviors. In this system, the BT executor receives dispatched missions from the orchestrator and translates them into navigation goals.

### 2.4 Python

Python was adopted as the primary implementation language of the proposed system, selected for its native compatibility with the ROS 2 ecosystem and its suitability for the varied computational requirements of each implemented layer.

### 2.5 Ollama

Ollama is used as the local inference engine for running the LLM (Llama 3.2) within the proposed system. It enables offline execution of the model with sub-second latency, which is essential for real-time robotic decision-making. This choice also ensures data privacy and system independence from external

cloud APIs. Ollama was selected for its simplicity, local deployment capability and minimal latency in response time.

## **2.6 Langchain**

LangChain is used as an application framework to manage interaction with the LLM. It handles prompt construction, model invocation, and response processing, providing a structured interface between user inputs and the inference engine.

## **2.7 Vision Language Model Qwen3**

Qwen3 is a Vision-Language Model (VLM), can simultaneously understand images and produce structured text descriptions of what it sees. Designed for embodied AI applications, it enables the robot to perceive and reason about its physical environment in real time. In this system, the model is deployed on the robot to continuously analyze camera frames, perform semantic analysis of the scene, classify the state of the path ahead (clear, blocked, or crowded), and locate observed obstacles in the map. The model was selected for two reasons. First, its 4-billion-parameter size makes it runnable on a consumer GPU through 4-bit quantization. Second, it demonstrates strong scene understanding in cluttered indoor environments, which is the primary deployment context of this project.

## **2.8 Milvus Db**

Milvus is a vector database used to store the spatio-temporal memory of the robot. It records past observations, including spatial position, event type, and timestamp, to support experience-informed navigation decisions. In addition, It is suitable for future extensions toward episodic memory and minimize migration cost.

## **2.9 Streamlit**

Streamlit is a Python framework for building interactive web applications. It was used to develop the mission control dashboard, providing a simple web interface where user can input commands. Streamlit was selected for its simplicity of use.

## **2.10 Gazebo**

Gazebo is a 3D robot simulation environment. It replicates the physical behavior of the robot, including its sensors, wheel dynamics, and interactions with the environment allowing the navigation stack and behavior tree to be tested in a virtual world before any deployment on real hardware. In this project, Gazebo was used throughout the execution layer to validate navigation edge cases without risk to the physical robot. Gazebo was selected because it integrates natively with ROS2 Jazzy.

## 2.11 Rviz 2

RViz 2 is the standard visualization tool of the ROS 2 ecosystem. It provides a real-time graphical view of the robot's internal state.

# III Implementation

In accordance with the Incremental Model methodology adopted for the proposed solution, this section details the four increments of the system's development. Each increment added working, testable components to what already existed and was validated before moving to the next. This approach solved a key problem : each component's behavior could only be fully understood after seeing how it worked with the previous one. A complete system validation is presented at the end of the chapter.

## 1 Increment 1 : Autonomous Navigation and Simulation Baseline

### 1.1 Objective

Establish the core navigation capabilities of the robot, enabling it to move autonomously within a dynamic indoor environment, avoid obstacles, and provide the foundation for all higher-level mission execution.

### 1.2 Implemented Components

#### 1. Simulation Setup and Environment Mapping and localization

For the simulation environment, an open-source hospital world was selected due to its suitability for dynamic and unpredictable environments.

For environment mapping, a two-dimensional (2D) map was generated using **SLAM Toolbox**. It was selected for its ability to perform simultaneous localization and mapping in real time while providing stable and reliable map generation within indoor environments. The generated map serves as the foundation for localization and autonomous navigation functionalities within the system. To maintain an estimate of the robot's pose within the generated map during autonomous operation, the Adaptive Monte Carlo Localisation (AMCL) maintains a continuous estimate of the robot's pose on the pre-built map through a particle filter of 500 to 2000 particles, dynamically scaled according to localisation confidence. The robotic platform used throughout this project is the ROSbot 2 Pro. Its Technical specifications are provided in Appendix 1.

#### 2. Navigation Stack

Nav2 serves as the core navigation framework, orchestrating global path planning, local trajectory execution, and obstacle avoidance. The stack operates on a costmap representation derived from the static map and real-time sensor data.

The costmap is a two-dimensional grid where each cell is assigned a value representing its navigability. The Table 5.2 summarizes the interpretation of these values.

TABLE 5.2 – Costmap Value Interpretation

| Cost  | Meaning         | Application                                   |
|-------|-----------------|-----------------------------------------------|
| 0     | Free space      | Preferred traversal area                      |
| 1–253 | Graduated cost  | Areas with increasing traversal difficulty    |
| 254   | Lethal obstacle | Impassable regions such as walls or furniture |
| 255   | Unknown         | Areas not yet observed by sensors             |

Two costmap layers are maintained :

- **Global costmap** : constructed from the static map and used for global path planning.
- **Local costmap** : updated in real time using LiDAR sensor data to handle dynamic obstacle avoidance.

The interaction between the two costmap layers enables both global path planning and local reactive obstacle avoidance.

For the proposed solution, the following global and local planners were adopted, respectively.

- **Dijkstra’s algorithm**

The global planner computes an optimal path between the robots current position and the target goal. Dijkstra’s algorithm is used for its ability to find the shortest path in terms of accumulated traversal cost. It therefore produces a lowest-cost path over the costmap representation. The search is performed over the costmap grid by progressively expanding nodes from the start position until the goal is reached.

- **Model Predictive Path Integral (MPPI) controller**

Model Predictive Path Integral (MPPI) is a sampling-based optimal control algorithm used for nonlinear systems. It generates 10,000 candidate trajectories by sampling control inputs, evaluates them using a cost function, and selects the optimal control sequence based on a weighted combination of the lowest-cost trajectories. This approach enables real-time, smooth, and collision-free local trajectory execution in dynamic environments.

### 3. Behavior Trees Design for Mission Sequencing

To establish the execution backbone of the system and prepare the robot for mission acceptance from the operator. We designed three Behavior Trees branches. Each branch corresponds to a distinct task type and exercises the core mechanisms of task sequencing, conditional triggering, and state preservation across preemption boundaries. The tree adopts a priority-based selector, ensuring that higher-priority branches preempt lower-priority ones whenever their entry conditions are satisfied. The priority order, from lowest to highest is :

- **Battery Sequence** This branch triggers autonomous docking . The robot navigates to the dock, aligns using AprilTag, and recharges
- **Mission Sequence** This branch handles basic GoTo commands accepted from the operator. It executes a single navigation step to the target location.

- **Delivery Sequence** This branch handles transport tasks requiring multi-step execution. It executes four stages : navigate to pickup, confirm load, navigate to drop-off, confirm delivery.

## 2 Increment 2 : Mission Parsing and Feasibility Evaluation

### 2.1 Objective

The system must accomplish two things in sequence : understand what the user is asking, then decide whether the robot has sufficient energy to comply and evaluate their feasibility against battery and safety constraints, producing accept, decline, or counter-proposal verdicts.

### 2.2 Implemented Components

#### 1. Natural Language Mission Parser

user submit mission through the web interface then The parser uses Llama 3.2 via Ollama with temperature  $T = 0$  for deterministic output guaranteeing deterministic output : identical inputs produce identical parsed results. This is essential for a safetycritical system. It converts natural language into a structured JSON command containing task intent such as action type, locations.

##### Prompt Architecture

Parser correctness depends primarily on prompt design. The pre-prompt is decomposed into five functionally distinct components, each targeting a specific failure class identified during prototyping :

- **Role** : Assigns the LLM its parsing role, restricting output to JSON only.
- **Environment** : Supplies the closed vocabulary of 26 valid location keys with bilingual aliases, preventing invalid map identifiers.
- **Action Rules** : Defines all six action types, required fields, verb-inheritance.
- **Example** : Provides an output exemple.

The prompt injected at every inference call is presented in Appendix 2.

#### 2. Energy Cost Model

Before accepting any mission, the system must predict how much battery energy it will consume. The energy model was therefore designed to use the actual path that the navigation stack would plan, not a simple straight-line distance, accounting for the curvature and any obstacles on the costmap. This component is divided into the following parts :

##### — Path Length via the Nav2 Navigation Stack

Rather than estimating distances geometrically, the system queries the Nav2 navigation stack directly to obtain the true planned path length for each navigation leg. The query follows the standard ROS 2 action communication pattern. A `ComputePathToPose` action client is created at startup. When a cost estimate is needed, the client sends an asynchronous goal to the global path planner, which runs Dijkstra’s algorithm on the current costmap. The goal specifies two poses : the start the robot’s current position as reported by AMCL (the

localisation module that continuously estimates the robot's position on the map by matching live LiDAR scans against the pre-built occupancy grid) and the destination waypoint. Then Dijkstra computes the globally optimal trajectory and returns the full sequence of poses along the computed path. Therefore, The total path length  $d$  is computed as the sum of the Euclidean distances between successive poses along the path, as expressed in Equation (5.1) :

$$d = \sum_{k=1}^{m-1} \sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2} \quad (5.1)$$

where  $m$  denotes the number of poses in the path, and  $(x_k, y_k)$  represents the coordinates of the  $k$ -th pose in the map frame.

#### — Energy Cost Formulas

Given the planned path length  $d$  obtained from the Nav2 query, the base energy cost of a single navigation is expressed in Equation (5.2) :

$$C_{\text{base}} = d \cdot \alpha + \sigma \quad (5.2)$$

where :

- $\alpha = 0.30\%/m$  is the empirically calibrated energy drain rate.
- $\sigma = 2.0\%$  is a fixed safety margin.

For a delivery task, the robot must travel from its current position to a pickup waypoint, then from the pickup to the drop-off waypoint. The total cost is therefore the sum of two independent navigation costs, as expressed in Equation (5.3) :

$$C_{\text{delivery}} = C(\text{start} \rightarrow \text{pickup}) + C(\text{pickup} \rightarrow \text{dropoff}) \quad (5.3)$$

#### — Battery Cursor Model

The battery cursor  $\Gamma$  tracks the projected battery level at the end of the entire current task queue. Let  $B$  denote the actual battery reading (percentage) from the ROS 2 battery topic, and  $Q = \{t_1, t_2, \dots, t_n\}$  the ordered queue. The cursor is formally defined in Equation (5.4) :

$$\Gamma = B - \sum_{i=1}^n C_i \quad (5.4)$$

where each  $C_i$  is the energy cost of task  $t_i$ , computed from its correct start position that is, the endpoint of the preceding task  $t_{i-1}$ . The cursor is fully rebuilt after every queue modification (addition, removal, reordering, or task completion), with the running task's remaining distance computed from the live AMCL pose.

For **delivery tasks**, feasibility is evaluated against the actual battery  $B$ , because deliveries preempt the entire queue and execute immediately.

For all other tasks (goto, chain steps), feasibility is evaluated against the cursor  $\Gamma$ , preven-

ting double counting of energy already committed to earlier tasks in the queue.

### 3. Deterministic Feasibility Decision Engine

Let  $\Gamma_{\text{eff}}$  denote the effective battery :

$$\Gamma_{\text{eff}} = \begin{cases} B & \text{for delivery tasks (preempt the entire queue)} \\ \Gamma & \text{for goto, chain tasks} \end{cases}$$

Let  $C_{\text{task}}$  be the estimated task cost from the energy model, and  $F = 35.0\%$  the hard battery floor. The decision engine applies three sequential checks :

DECLINE if  $\Gamma_{\text{eff}} < F$  (hard floor violation)

DECLINE if  $\Gamma_{\text{eff}} - C_{\text{task}} < 0$  (cost exceeds available energy)

ACCEPT otherwise

For a Counter-Proposal case when a non-delivery task receives a DECLINE but would become feasible after recharging, a COUNTER verdict is issued. The task is automatically promoted to the active queue after the Behavior Tree fires a dock complete event.

## 3 Increment 3 : Dynamic Task Orchestration

### 3.1 Objective

Once a mission is accepted, the system must decide how to integrate it into a live, evolving schedule alongside tasks that are already running, queued, or suspended. The outcome is a robot that adapts its workload autonomously, without operator intervention, as new missions arrive and conditions change.

### 3.2 Implemented Components

#### 1. Task Lifecycle and Queue States

Every task in the queue carries a status field that governs how the orchestrator treats it. The six possible states are : running , queued,interrupted, counter\_proposed, done and cancelled .

#### 2. Proximity-Ordered Task Queue

Every time a task is accepted, the full pending queue is reordered using a nearest-neighbour algorithm : starting from the endpoint of the running task, the system repeatedly selects the closest destination until all tasks are placed. The result minimises total travel distance across the pending workload. Energy costs and the battery cursor are then rebuilt in full from each task's updated start position, ensuring that feasibility evaluations always operate on accurate data.

#### 3. The High-Priority Override

Delivery tasks carry the highest priority. When a delivery is accepted, the orchestrator does not queue it behind existing work.It promotes it to immediate execution. All running and queued

tasks are suspended. Before execution begins, the system runs a resumption preview : starting from the delivery’s dropoff waypoint and using the estimated post-delivery battery, it simulates the cost of resuming each interrupted task in nearest-first order, flagging each as resumable or cancelled after delivery before the robot moves.

#### 4. On-the-Way Interrupt

When a newly accepted task’s destination is closer than the robot’s current target, executing it first eliminates unnecessary backtracking. The orchestrator pauses the running task, dispatches the closer task immediately, and computes the interrupted task’s resumption cost from the new task’s endpoint. The original task is re-dispatched automatically upon completion.

## 4 Increment 4 : Memory builder and Injection

### 4.1 objective

This increment equips the robot with a spatio-temporal memory, a growing record of environmental observations, specifically blocked and crowded corridors, that the Decision Layer can query before dispatching any mission, injecting past knowledge directly into the navigation stack’s costmap before the robot moves.

### 4.2 Implemented Components

#### 1. Vision-Language Scene Analysis

The system adopts **Qwen3-VL-2B-Instruct** as the perception backbone. The model processes each camera frame through a two-pass inference pipeline to extract both scene classification and obstacle localisation.

##### — Two-Pass Inference Pipeline

Each camera frame triggers a two-pass inference sequence. The passes are architecturally independent. Pass 1 classifies the scene, Pass 2 localises the obstacle within it and only Pass 2 is executed when Pass 1 identifies an obstacle status that warrants spatial grounding.

##### — Pass 1 Scene Classification

The prompt is structured into six sections : ROLE, ENVIRONMENT, Attentive Reasoning Queries (ARQ), Status Decision Rules, Output Schema, and Calibrated Examples. The ARQ section forces the model to resolve targeted sub-questions internally before producing its output, addressing known VLM failure modes in indoor navigation scenes. The Pass 1 classification prompt is provided in Appendix 3.

### — Pass 2 Obstacle Grounding

When Pass 1 returns `path_status = blocked` a second inference call locates the obstacle. The Pass 2 grounding prompt is provided in Appendix 4.

The returned normalised coordinates ( $\text{norm}_x, \text{norm}_y$ ) By the VLM are then converted to pixel coordinates  $(u, v)$  using the image dimensions  $W$  (width) and  $H$  (height) in pixels, as expressed in Equation (5.5) :

$$u = \text{round}\left(\frac{\text{norm}_x}{1000} \times W\right) \quad v = \text{round}\left(\frac{\text{norm}_y}{1000} \times H\right) \quad (5.5)$$

where  $u$  and  $v$  denote the horizontal and vertical pixel coordinates respectively in the image plane.

## 2. Camera Calibration and Obstacle Localisation

This section describes the geometric pipeline that transforms a VLM-localised pixel into a global map coordinate using three sequential transforms : depth estimation, pinhole back-projection, and coordinate frame alignment.

### — Depth Estimation

The robot’s OAK-D stereo camera provides a depth map aligned with each RGB frame. Taking advantage of this sensor, the system directly obtains a depth estimate  $d$  at the grounded pixel  $(u, v)$ .

### — Pinhole Back-Projection

With the depth estimate  $d$ , the system projects the pixel into the 3D camera frame. The OAK-D’s intrinsic parameters are listed in Table 5.3.

TABLE 5.3 – OAK-D Camera Intrinsic Parameters

| Parameter        | Value      | Description               |
|------------------|------------|---------------------------|
| $f_x = f_y$      | 1108.51 px | Focal length              |
| $c_x$            | 640.00 px  | Horizontal optical centre |
| $c_y$            | 360.00 px  | Vertical optical centre   |
| $W_{\text{img}}$ | 1280 px    | Image width               |

The back-projection formula places the obstacle in the camera frame ( $X$  right,  $Y$  down,  $Z$  forward), as expressed in Equation (5.6) :

$$X_{\text{cam}} = \frac{(u - c_x) \cdot d}{f_x}, \quad Z_{\text{cam}} = d \quad (5.6)$$

The vertical coordinate  $Y_{\text{cam}}$  is discarded. For a floor obstacle, only the horizontal offset and forward distance matter for 2D navigation.

### — Camera-to-Robot-to-Map Transform

The camera frame differs from the robot’s navigation frame. so the transform from camera to robot frame is expressed in Equation (5.7) :

$$X_{\text{robot}} = Z_{\text{cam}}, \quad Y_{\text{robot}} = -X_{\text{cam}} \quad (5.7)$$

Using the AMCL-reported pose  $(x_r, y_r, \theta)$ , the robot frame coordinates are then rotated and translated into the global map frame via the standard 2D rigid-body transform, as expressed in Equations (5.8) and (5.9) :

$$X_{\text{map}} = x_r + X_{\text{robot}} \cos \theta - Y_{\text{robot}} \sin \theta \quad (5.8)$$

$$Y_{\text{map}} = y_r + X_{\text{robot}} \sin \theta + Y_{\text{robot}} \cos \theta \quad (5.9)$$

The resulting pair  $(X_{\text{map}}, Y_{\text{map}})$  constitutes the 2D obstacle position expressed in metres in the global map frame.

With  $(X_{\text{map}}, Y_{\text{map}})$  established, a linear barrier of lethal costmap points is constructed perpendicular to the robot’s heading  $\theta$ , as expressed in Equation (5.10) :

$$p_{\perp} = (-\sin \theta, \cos \theta) \quad (5.10)$$

The barrier width is computed directly from the stereo depth  $d$  using the camera’s horizontal Field of View (FOV), as expressed in Equation (5.11) :

$$w = \frac{W_{\text{img}}}{f_x} \cdot d \quad (5.11)$$

### 3. Memory Record Schema

In the Milvus database, we store every observation the robot makes. Each record captures four essential pieces of information : where the robot was (we store robot position pos\_x, pos\_y, and heading theta), when the observation occurred (we store time), what the VLM classified, and where the obstacle is located in the map.

The complete schema is presented in Table 5.4.

TABLE 5.4 – Milvus Memory Record Schema

| Field                | Type   | Description                                                                                                                                 |
|----------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------|
| path_id              | String | Robot’s current path                                                                                                                        |
| path_status          | String | VLM classification : <code>clear</code> , <code>semi_blocked</code> , <code>blocked</code> , <code>crowded</code> , or <code>unknown</code> |
| confidence           | String | VLM certainty                                                                                                                               |
| pos_x, pos_y         | Float  | Robot AMCL position in global map frame                                                                                                     |
| theta                | Float  | Robot heading                                                                                                                               |
| obs_map_x, obs_map_y | Float  | Projected obstacle position in map frame                                                                                                    |
| obs_depth_m          | Float  | Stereo depth to obstacle centre                                                                                                             |
| timestamp_unix       | Double | Unix timestamp of observation (seconds)                                                                                                     |
| caption              | String | Rich one-sentence retrievable description for future LLM queries                                                                            |

#### 4. Memory Injection into the Navigation Stack

After building the context from recent observations, a query is triggered before each mission is dispatched to the Behavior Tree. The retrieved events are translated into costmap modifications and injected directly into the navigation stack’s global costmap, influencing path planning before the robot moves.

This is achieved through two sequential steps : first, querying the memory, second injecting the results into the costmap.

- **Memory Retrieval** The query applies a scalar filter on the Milvus database, combining `path_status` and recency to retrieve observations.

Two query types are supported :

- **Blocked positions** : retrieves a list of  $(x, y)$  points where `path_status == "blocked"`.
- **Crowded paths** : retrieves a list of corridor names where `path_status == "crowded"`.

#### 5. Two-Tier Injection

The classification of each retrieved record determines which injection strategy is applied to the navigation stack costmap, as shown in Table 5.5.

TABLE 5.5 – Memory Injection Types and Costmap Effects

| Status  | Injection Type  | Costmap Effect                             |
|---------|-----------------|--------------------------------------------|
| blocked | Lethal obstacle | Cost = 254, planner routes around the area |
| crowded | Keepout zone    | planner prefers alternative paths.         |

The two injection types differ in how they reach the navigation stack. For a blocked observation, the barrier points constructed are published as point cloud data describing the obstacle’s position and width in the map frame. The navigation stack consumes this point cloud and marks the corresponding costmap cells as lethal, forcing the global planner to route strictly around the affected area.

For a crowded observation, the system instead relies on a keepout filter plugin, which takes as input a mask matching the shape of the affected area and applies a non-lethal cost over that region. This cost is high enough to bias the planner toward alternative routes, while remaining below the lethal threshold so the area stays traversable if no other path exists.

## IV Validation

This section validates the system’s core functionalities through interface visualisations.

### 1 Environment Mapping

the hospital environment was first modelled in Gazebo. Figure 5.1 presents the 3D simulation environment used throughout the evaluation, including patient rooms, corridors, waiting areas.



FIGURE 5.1 – 3D visualisation of the hospital simulation environment

Using SLAM, the robot constructs a 2D representation of the environment that serves as the basis for autonomous navigation. Figure 5.2 shows the costmap generated by the navigation stack from the mapped environment. Occupied regions correspond to walls and obstacles, while the inflated areas around obstacles define safety margins used during path planning.

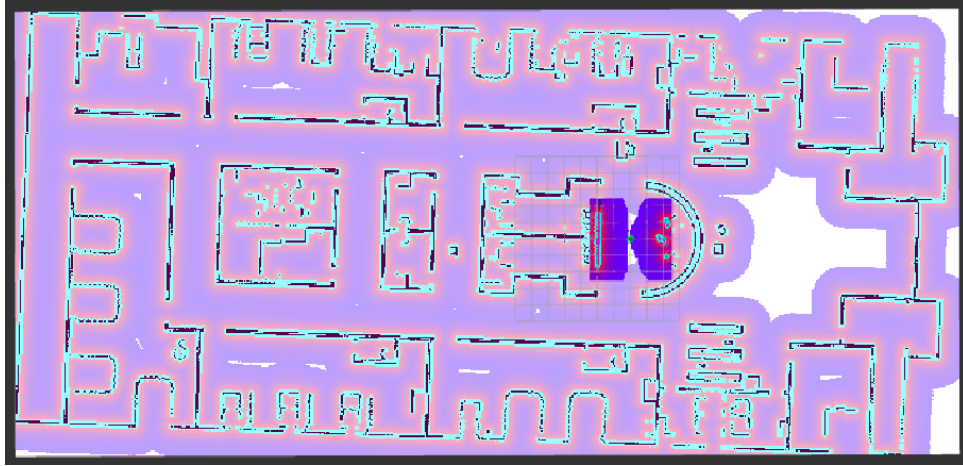


FIGURE 5.2 – Costmap generated by the navigation stack for autonomous navigation and path planning

## 2 Proximity-Based Task Reordering Validation

A chain mission was submitted with the instruction. The system automatically reordered the sequence based on the robot's current position.

Figure 5.3 shows the User Interface (UI) response, where the three steps are presented in the optimised ordere.

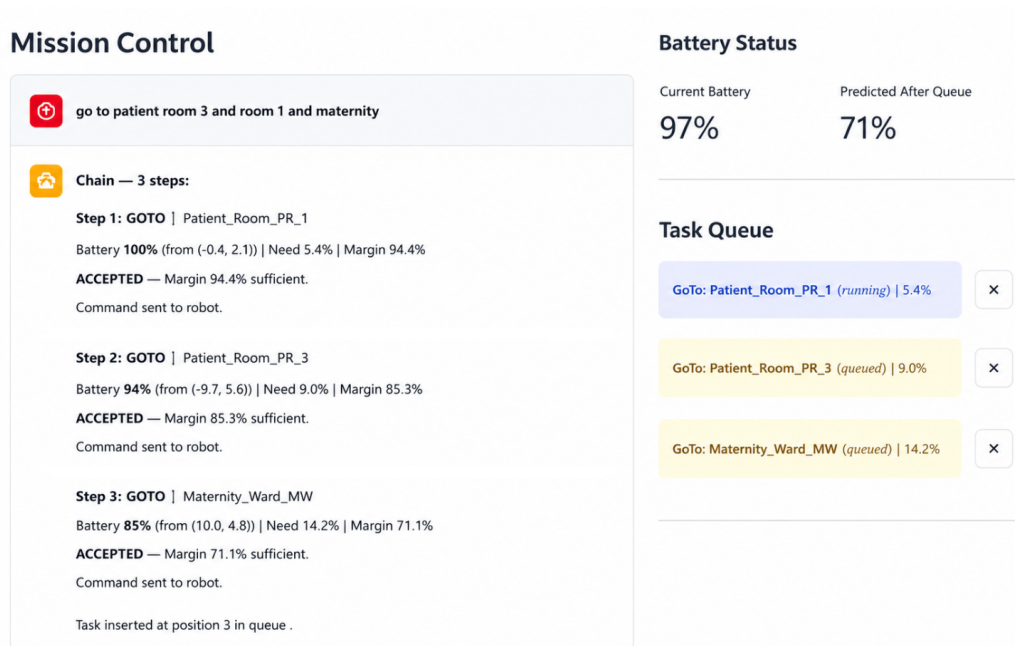


FIGURE 5.3 – UI response showing reordered chain mission based on proximity

As shown in Figure 5.3 response, each step includes the predicted battery consumption, remaining margin, and confirmation that the command has been sent to the robot. The system successfully inserted the optimised sequence into the task queue.

### 3 high-priority Interrupt Validation

To validate the preemption mechanism for high-priority tasks and the resumption of interrupted tasks a delivery mission was submitted.

Figure 5.4 shows the UI response during the preemption event.

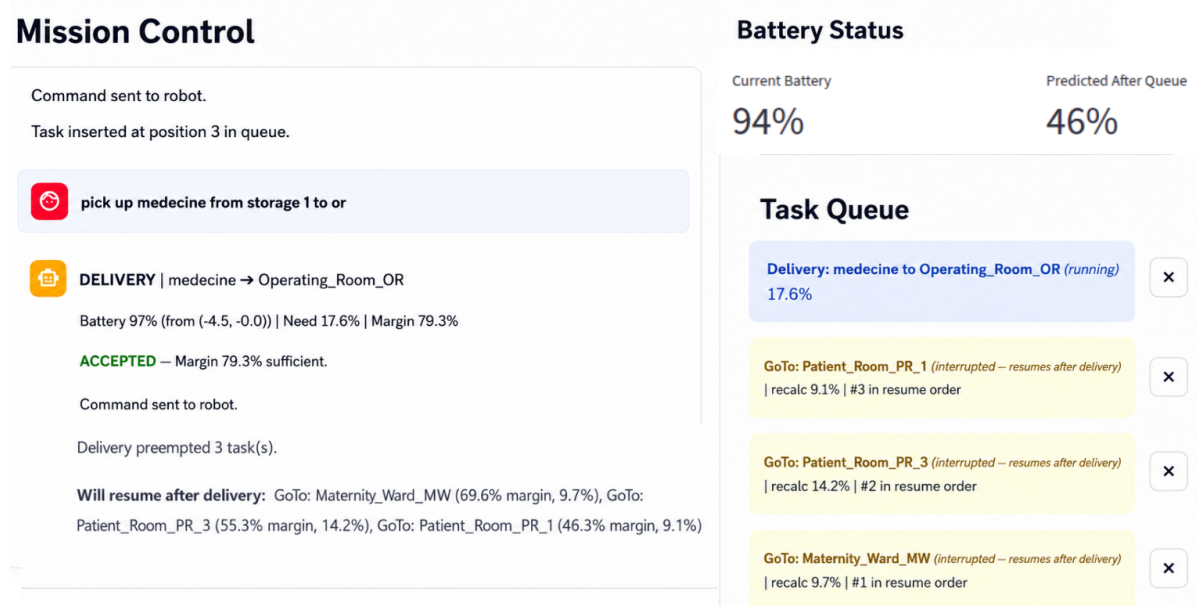


FIGURE 5.4 – UI response showing delivery preemption with resume order

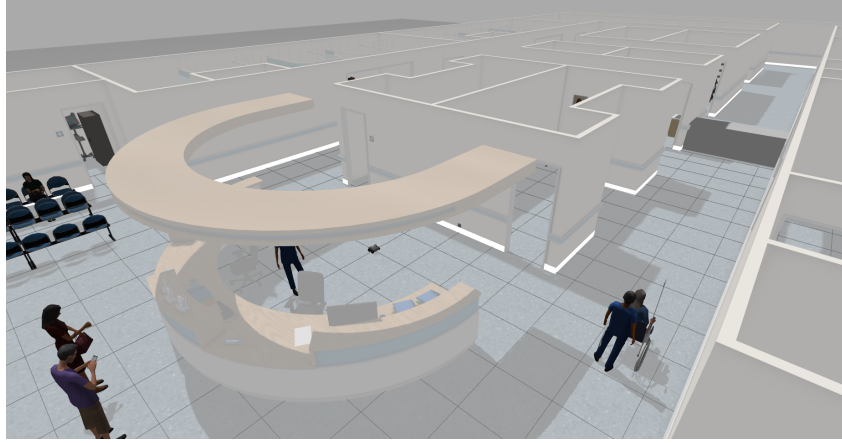
As shown in Figure 5.4, the delivery preempted 3 existing tasks. After delivery completion, the interrupted tasks were reordered from the dropoff waypoint using a nearest-first proximity algorithm, with each task's energy cost recalculated based on the new starting position.

### 4 Obstacle Injection Validation

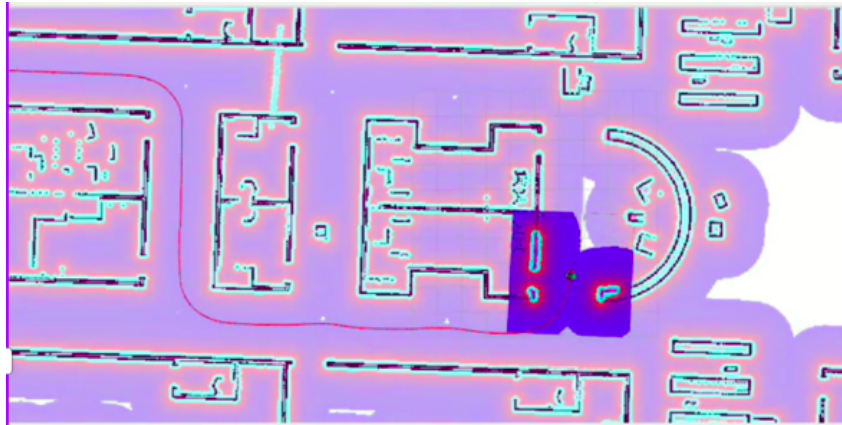
The memory injection mechanism is validated by recording two types of corridor conditions : blocked and crowded. For each condition, the 3D visualisation shows the observed scenario, and the corresponding Navigation Stack costmap shows the injected result.

#### 4.1 Blocked Corridor

Figure 5.5 illustrates a corridor fully blocked by an obstacle spanning the entire width. The corresponding costmap shows the injected lethal obstacle.



(a) 3D view of a fully blocked corridor

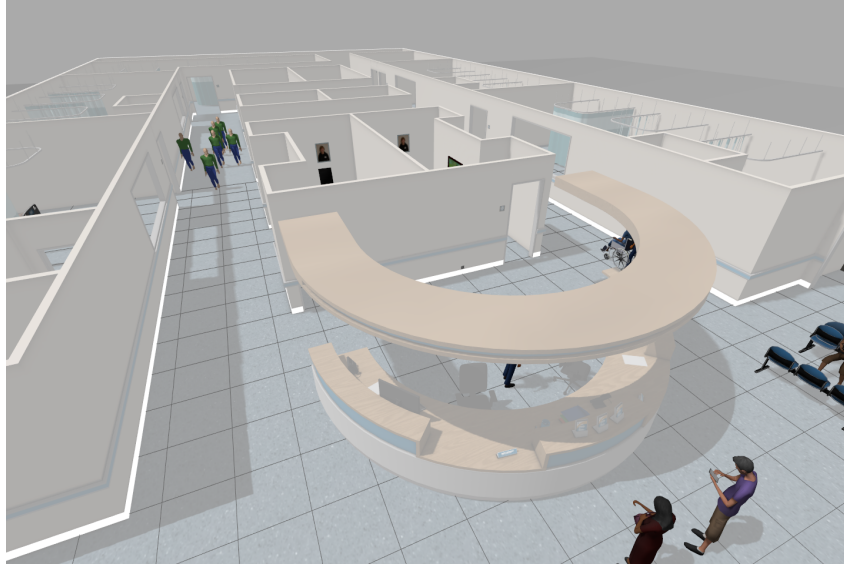


(b) Costmap after lethal obstacle injection

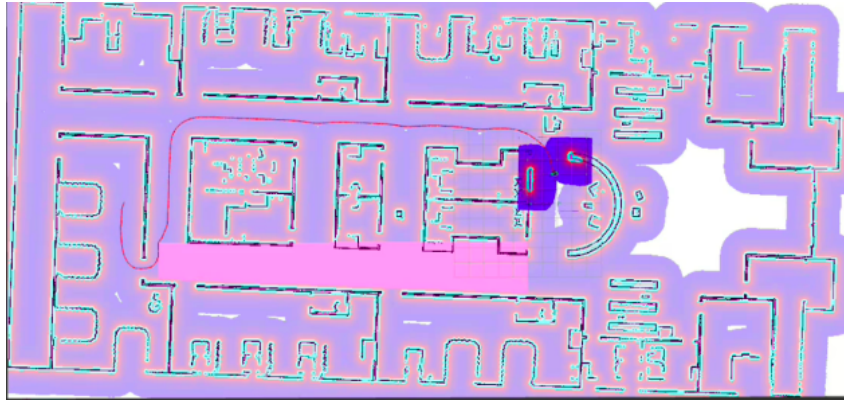
FIGURE 5.5 – Blocked corridor scenario and corresponding costmap modification.

## 4.2 Crowded Corridor

Figure 5.6 illustrates a corridor occupied by multiple pedestrians. The corresponding costmap shows the injected region, encouraging the navigation stack to avoid the area.



(a) 3D view of a crowded corridor



(b) Costmap after high-cost zone injection

FIGURE 5.6 – Crowded corridor scenario and corresponding costmap modification.

## V Conclusion

This chapter detailed the practical realisation of the proposed system, presenting the development environment, the tools used and the rationale behind their selection, together with the implementation work carried out across all system components through four incremental . Moreover results were provided to validate the proposed system.

# General Conclusion

This project, conducted at **Enova Robotics**, addressed the design and implementation of a task orchestration system for an autonomous mobile robot. Three objectives structured the work : building an environmental awareness layer, implementing a resource-conscious feasibility engine, and developing a dynamic priority management mechanism for multi-task execution.

The system achieves a principled separation of concerns across three complementary layers, equipping the robot to interpret natural language instructions, evaluate mission feasibility against live battery constraints, manage a priority-ordered task queue under preemption, and navigate with accumulated environmental knowledge.

To satisfy the project requirements, a natural language parsing pipeline was built to interpret operator commands and produce structured task descriptors. A battery- aware feasibility engine was implemented to assess mission viability in real time and generate counter-proposals when the available charge was insufficient. A priority-ordered queue was developed to manage concurrent missions, handling preemption, nearest-neighbour reordering, and task resumption after interruption. A spatio-temporal memory pipeline was constructed to capture and store environmental observations. All these components were integrated and validated on the ROSbot 2 Pro, confirming that the system meets its functional requirements across the full mission lifecycle.

The work presented one main limitation. The computational capacity of the embedded platform required trade-offs between inference quality and response latency, particularly in the vision-language pipeline.

Beyond the technical outcomes, this internship sharpened the ability to make principled design decisions under real constraints choosing the right architecture, balancing competing requirements, and reasoning about trade-offs at every level of the system.

The present work opens several natural directions for future investigation. Semantic and episodic memory retrieval would enable context-aware queries by meaning, allowing the robot to learn recurring patterns from its episodic history such as recognising that a given corridor is consistently congested at certain hours and incorporate that knowledge into future scheduling decisions. Furthermore, multi-robot coordination and task allocation would extend the feasibility engine to a fleet, distributing missions across agents based on position, battery state, and current workload.

# Bibliography

- [1] GeeksforGeeks, *Incremental Process Model – Software Engineering*, Available at : <https://www.geeksforgeeks.org/software-engineering/software-engineering-incremental-process-model/>, 2025.
- [2] Ganney P.S., Claridge E., *Software Engineering : Incremental Model* z, dans *Clinical Engineering*, 2<sup>e</sup> éd., Academic Press, 2020. Available at : <https://www.sciencedirect.com/topics/computer-science/incremental-model>
- [3] Husarion, *ROS2 Introduction Tutorial*, Available at : <https://husarion.com/tutorials/ros2-tutorials/1-ros2-introduction/>, 2025.
- [4] Husarion, *ROS2 SLAM Tutorial*, Available at : <https://husarion.com/tutorials/ros2-tutorials/8-slam/>, 2025.
- [5] Macenski S., Jambrecic I., *SLAM Toolbox : SLAM for the dynamic world* z, *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021. Available at : <https://doi.org/10.21105/joss.02783>
- [6] Husarion, *ROS2 Navigation Tutorial*, Available at : <https://husarion.com/tutorials/ros2-tutorials/9-navigation/>, 2025.
- [7] *Navigation Stack Components*, Available at : <https://deepwiki.com/ros-navigation/navigation2/2-core-navigation-components>, 2025.
- [8] GeeksforGeeks, *Odometry in Machine Learning*, Available at : <https://www.geeksforgeeks.org/machine-learning/odometry/>, 2025.
- [9] Advanced Navigation, *Inertial Measurement Unit (IMU) – An Introduction*, Available at : <https://www.advancednavigation.com/tech-articles/inertial-measurement-unit-imu-an-introduction/>, 2025.
- [10] Menon D., *Robot Localization : EKF, AMCL and ROS 2 Sensor Fusion*, RoboCloud Hub, 2025. Available at : <https://robocloud-dashboard.vercel.app/learn/blog/robot-localization>
- [11] Existential Robotics, *Differential Drive Kinematics*, Available at : [https://existentialrobotics.org/RobotProvingGrounds/algorithms/control/content/diffy\\_drive/](https://existentialrobotics.org/RobotProvingGrounds/algorithms/control/content/diffy_drive/), 2025.
- [12] Spiceworks, *What is a Finite State Machine ?*, Available at : <https://www.spiceworks.com/soft-tech/what-is-fsm/>, 2025.

- [13] ResearchGate, *Robot Finite State Machine* [Figure], Available at : [https://www.researchgate.net/figure/Robot-finite-state-machine\\_fig2\\_358116166](https://www.researchgate.net/figure/Robot-finite-state-machine_fig2_358116166), 2025.
- [14] Little Polygon, *Finite State Machines*, Available at : <https://blog.littlepolygon.com/posts/fsm/>, 2025.
- [15] Robohub, *Introduction to Behavior Trees*, Available at : <https://robohub.org/introduction-to-behavior-trees/>, 2025.
- [16] Husarion, *ROSbot 2 Pro Datasheet*, Available at : <https://husarion.com/manuals/rosbot/>, 2025.

# Appendices

## Appendix 1 : ROSbot 2 Pro Technical Specifications

The robotic platform used throughout this project is the ROSbot 2 Pro, developed by Husarion. Its technical specifications are summarised in Figure A.1 [16].

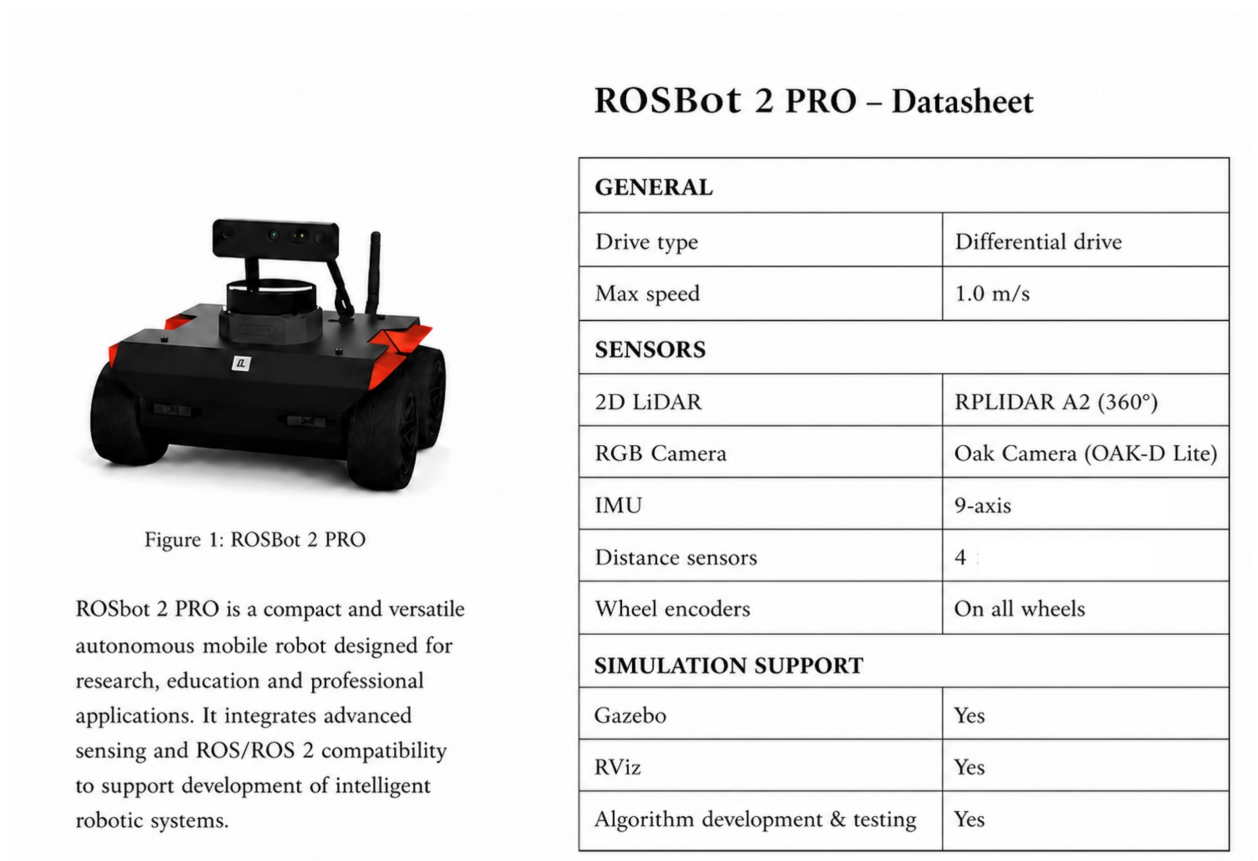


FIGURE A.1 – Rosbot 2 pro Datasheet.

## Appendix 2 : Mission Parser EXTRACTION PROMPT

An extract of The EXTRACTION PROMPT injected into the parser at every inference call is presented in Figure B.1.

### [ROLE]

You are the command parser for an autonomous hospital robot.

Parse ONLY the user's current message. History is context only.

### [LOCATIONS] -- 26 navigable locations, exact keys required

Clinical : Intensive\_Care\_Unit\_ICU, Operating\_Room\_OR,

Patient\_Room\_PR\_1..PR\_4, Maternity\_Ward\_MW ...

Support : Front\_Desk\_FD, Storage\_Room\_SR\_1/2, Visitor\_Waiting\_VW

Corridors: Main\_Hall, Corridor\_1 ... Corridor\_6

Aliases : réa->ICU | bloc->OR | pharmacie->SR\_1 | chambre 2->PR\_2 ...

### [ACTION TYPES]

delivery : item transport -- location\_a + item + location\_b

goto : navigate only -- room

chain : 2+ actions -- steps[]

unknown : parse failure

### [RULES]

-- VERB INHERITANCE : 'goto PR\_1 and PR\_3' -> both inherit 'goto'

-- DELIVERY : explicit item name required; no item -> NOT delivery

### [OUTPUT EXAMPLE]

```
{
  "action"      : "delivery",
  "location_a": "Storage_Room_SR_1",
  "item"        : "medication",
  "location_b": "Intensive_Care_Unit_ICU"
}
```

FIGURE B.1 – LLM Extraction Prompt

## Appendix 3 : Vision-Language Model Pass 1 Scene Classification Prompt

The Pass 1 prompt is injected into Qwen3-VL-4B-Instruct at every camera frame inference call. It governs scene status classification through structured Attentive Reasoning Queries (ARQ) and produces a JSON memory record. An extract of The Pass 1 classification prompt is presented in Figure C.1.

```
=== ROLE ===

You are the vision system of a hospital security patrol robot.
Your output is a structured memory record stored in the robot's database.
Accuracy is the only acceptable standard. Never guess. Never hallucinate.

=== ENVIRONMENT ===

Indoor hospital corridors, wards, waiting areas.
Camera faces direction of travel.

=== ARQ (Answer INTERNALLY, never write) ===
Q1: Scan CENTER FLOOR only. Any object on the floor?
Q2: Wall Anchor Test: Are BOTH corridor walls visible alongside the obstacle?
    YES  semi_blocked | NO  blocked
Q3: Count people. Seated at wall = no path impact. Standing in center = apply Q2.
Q4: Confidence: high / medium / low based on image quality.

=== STATUS DECISION RULES ===

clear          Nothing on center floor.
semi_blocked   Both walls visible alongside obstacle.
blocked        Walls NOT both visible.
crowded        4+ people standing in travel zone.
unknown        Image too degraded.

=== OUTPUT SCHEMA ===
{
  "people_count": <int>,
  "path_status": "<clear|semi_blocked|blocked|crowded|unknown>",
  "confidence": "<high|medium|low>",
  "caption": "<retrievable sentence>"
}
```

FIGURE C.1 – VLM pass 1 Prompt

## Appendix 4 : Vision-Language Model Pass 2 Obstacle Grounding Prompt

The Pass 2 grounding prompt, conditionally triggered when Pass 1 returns `path_status = blocked`. It instructs the model to return the normalised point coordinates of the primary floor obstacle, which are subsequently converted to pixel coordinates for costmap injection. The Pass 2 grounding prompt is presented in Figure D.1.

```
Locate the obstacle blocking the corridor floor.  
Output the point coordinates in JSON format.
```

```
Example:
```

```
[{"point_2d": [523, 681], "label": "obstacle"}]
```

```
Where x and y are integers in [0, 1000].
```

```
(0,0) = top-left, (1000,1000) = bottom-right.
```

FIGURE D.1 – VLM pass 2 prompt